

■ React 19 Interview Preparation Guide

100 MCQ Questions with Explanations & Code Examples

Intermediate Level | All Hooks Covered | React 19 Features
Real-Life Examples | Technical Deep-Dive

Section	Topic	Questions
1	React 19 New Features	1–5
2	Core Hooks: useState, useEffect, useContext	6–10
3	useRef, useMemo, useCallback	11–15
4	useReducer	16–20
5	useTransition & useDeferredValue	21–25
6	useId & useImperativeHandle	26–28
7	Custom Hooks	29–32
8	React 19 Actions & Forms	33–35
9	React.memo, Lazy & Suspense	36–40
10	Performance Optimization	41–45
11	Context API Deep Dive	46–48
12	Portals & Error Boundaries	49–50
13	React 19 Asset Loading	51–52
14	Hooks Rules & Patterns	53–56
15	Advanced Patterns	57–66
16	Popular Hook Packages	71–75
17	React 19 Deep Dive	76–80
18	Testing Basics	81–83
19	React Router v6	84–86
20	Mixed / Final Round	87–100

SECTION 1: React 19 – New Features & Changes

1. React 19 mein `forwardRef` ke baare mein kya badla hai?

- A) forwardRef aur bhi required ho gaya
- B) forwardRef completely remove ho gaya, ab ref directly prop ki tarah pass hota hai
- C) forwardRef ka naam refForward ho gaya
- D) forwardRef sirf class components mein kaam karta hai

✓ Answer: B

■ Why?

React 19 mein ab aapko forwardRef() wrapper likhne ki zaroorat nahi. ref directly function component ko prop ki tarah mil jaata hai — bilkul jaise onClick ya className milta hai.

```
// React 18 (purana tarika)
const Input = forwardRef((props, ref) => (
  <input ref={ref} {...props} />
));

// React 19 (naya tarika) ■
function Input({ ref, ...props }) {
  return <input ref={ref} {...props} />;
}
```

■ Real Life: Ek design system mein aap Button, Input, Modal — sab mein forwardRef likhte the. Ab seedha ref pass karo, code 30% chhota ho jaata hai.

2. React 19 mein `use()` hook kya karta hai?

- A) Sirf custom hooks banane ke liye
- B) Promise aur Context dono ko conditionally read karne deta hai
- C) useEffect ka replacement hai
- D) Server-side rendering ke liye sirf

✓ Answer: B

■ Why?

`use()` ek naya hook hai jo Promise aur Context dono ko read kar sakta hai. Sabse bada faida — yeh conditionally bhi call ho sakta hai, jo React ke purane hook rules ke against tha.

```
import { use, Suspense } from 'react';

function UserName({ userPromise }) {
  const user = use(userPromise); // Promise resolve karega
  return <h1>{user.name}</h1>;
}

// Context bhi use() se read kar sakte ho
function Theme({ children }) {
  const theme = use(ThemeContext); // useContext jaisa
  return <div className={theme}>{children}</div>;
}
```

■ Real Life: E-commerce app mein product details fetch karna — Suspense ke saath use() lagao, loading spinner automatically handle hoga.

3. React 19 mein `` ko provider ki tarah kaise use karte hain?

- A)
- B) directly
- C)
- D) useContext(Context, value)

✓ **Answer: B**

■ **Why?**

React 19 mein ab likhne ki zaroorat nahi. Aap seedha likh sakte ho. Yeh cleaner syntax hai.

```
const ThemeContext = createContext('light');

// React 18
<ThemeContext.Provider value="dark">
  <App />
</ThemeContext.Provider>

// React 19 ■
<ThemeContext value="dark">
  <App />
</ThemeContext>
```

■ **Real Life:** Puri app mein dark/light mode context wrap karna — naya syntax zyada readable aur concise hai.

4. React 19 mein `useOptimistic` hook ka main purpose kya hai?

- A) API call cancel karna
- B) UI ko optimistically update karna jab async operation chal raha ho
- C) Component ko optimize karna re-renders ke liye
- D) Lazy loading ke liye

✓ **Answer: B**

■ **Why?**

`useOptimistic` आपको UI ko turant update karne deta hai server response aane se pehle. Agar server fail ho, toh automatically rollback ho jaata hai.

```
import { useOptimistic } from 'react';

function LikeButton({ post }) {
  const [optimisticLikes, addOptimisticLike] = useOptimistic(
    post.likes,
    (current, newLike) => current + newLike
  );

  async function handleLike() {
    addOptimisticLike(1); // Turant +1 dikhao
    await likePost(post.id); // Server call
  }
  return <button onClick={handleLike}>♥■ {optimisticLikes}</button>;
}
```

■ **Real Life:** Instagram mein like karte ho — button turant fill ho jaata hai bina server response ka wait kiye. Yahi useOptimistic karta hai.

5. `useFormStatus` hook kahan se import karte hain React 19 mein?

- A) react
- B) react-dom

- C) react-dom/client
- D) react/hooks

✓ **Answer: B**

■ **Why?**

`useFormStatus` react-dom package mein hai, react mein nahi. Yeh hook form submission ka status deta hai — pending, data, method, action.

```
import { useFormStatus } from 'react-dom';

function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button disabled={pending}>
      {pending ? 'Submitting...' : 'Submit'}
    </button>
  );
}

function MyForm() {
  return (
    <form action={submitAction}>
      <input name="email" />
      <SubmitButton /> {/* Form ke andar hona chahiye */}
    </form>
  );
}
```

■ **Real Life:** Login form mein submit button ko disable karna jab tak request complete nahi ho — double submit prevent karta hai.

SECTION 2: Core Hooks – useState, useEffect, useContext

6. `useState` mein functional update kab use karni chahiye?

- A) Jab state string ho
- B) Jab naya state purane state par depend karta ho
- C) Sirf arrays ke liye
- D) Async operations mein sirf

✓ **Answer: B**

■ **Why?**

Jab naya state purane state ki value pe depend kare, toh functional update form use karo. Yeh stale closure problem solve karta hai — React guaranteed latest state deta hai callback mein.

```
// Galat tarika (stale closure ho sakta hai)
setCount(count + 1);

// Sahi tarika ■
setCount(prevCount => prevCount + 1);

// Example: Rapid clicks handle karna
function Counter() {
  const [count, setCount] = useState(0);
  function handleFastClick() {
```

```

setCount(c => c + 1); // Har click par latest value milegi
setCount(c => c + 1);
setCount(c => c + 1); // Result: +3
}
return <button onClick={handleFastClick}>{count}</button>;
}

```

■ Real Life: Shopping cart mein quantity badhana — rapid clicks pe sahi count chahiye, functional update guarantee deta hai.

7. `useEffect` ki dependency array mein empty array `[]` dene ka matlab kya hai?

- A) Effect har render pe chalega
- B) Effect bilkul nahi chalega
- C) Effect sirf mount pe chalega, unmount pe cleanup
- D) Effect sirf unmount pe chalega

✓ Answer: C

■ Why?

Empty dependency array `[]` matlab effect sirf component mount hone par ek baar chalega (componentDidMount jaisa). Agar cleanup function return karo, woh unmount par chalega.

```

useEffect(() => {
  console.log('Component mount hua'); // Sirf ek baar

  const subscription = subscribe(userId);

  return () => {
    subscription.unsubscribe(); // Unmount par cleanup
    console.log('Component unmount hua');
  };
}, []); // Empty array = sirf mount/unmount

```

■ Real Life: Chat app mein WebSocket connection banana on mount, aur user page chhodne par disconnect karna — yahi pattern use hota hai.

8. Niche diya code ka output kya hoga? `const [state, setState] = useState({name: 'Raj', age: 25});` `setState({name: 'Priya'})`;

- A) {name: 'Priya', age: 25}
- B) {name: 'Priya'}
- C) {name: 'Raj', age: 25}
- D) Error aayega

✓ Answer: B

■ Why?

useState mein setState purani state ko merge NAHI karta (class component ke this.setState se alag). Yeh state ko completely replace kar deta hai. Isliye spread operator use karo.

```

// Galat — age gayab ho jaayega
setState({ name: 'Priya' }); // Result: {name: 'Priya'}

// Sahi tarika ■
setState(prev => ({ ...prev, name: 'Priya' }));
// Result: {name: 'Priya', age: 25}

```

■ Real Life: User profile update karte waqt sirf email change karo, baaki fields preserve karo — spread operator zaroor lagao.

9. `useContext` re-render kab trigger karta hai?

- A) Jab koi bhi parent re-render ho
- B) Jab context ki value change ho
- C) Sirf jab component ke props change hoon
- D) Kabhi nahi, context re-render nahi karta

✓ Answer: B

■ Why?

useContext tab re-render trigger karta hai jab context provider ki value change ho. Agar value object hai aur har render pe naya object bane, toh unnecessary re-renders honge.

```
// Problem – har render pe naya object banta hai
<ThemeContext.Provider value={{ theme, setTheme }}>

// Solution – useMemo se stabilize karo ■
const contextValue = useMemo(
  () => ({ theme, setTheme }),
  [theme]
);
<ThemeContext.Provider value={contextValue}>
```

■ Real Life: Theme toggle app mein agar context value stabilize nahi kiya, toh puri app har render pe re-render hogi — performance issue.

10. `useEffect` mein cleanup function kab return karna zaroori hai?

- A) Sirf jab API call ho
- B) Jab bhi side effect ke baad resources clean karne ho (subscriptions, timers, event listeners)
- C) Sirf class components mein
- D) Kabhi zaroorat nahi

✓ Answer: B

■ Why?

Cleanup function tab return karo jab aapka effect koi resource allocate kare jaise timer, event listener, WebSocket, subscription. Bina cleanup ke memory leaks aur bugs aate hain.

```
useEffect(() => {
  // Timer set kiya
  const timerId = setTimeout(() => {
    setMessage('Time out!');
  }, 3000);

  // Cleanup – component unmount ya dependency change par
  return () => clearTimeout(timerId);
}, []);
```

■ Real Life: Auto-logout timer — agar user page chhodta hai cleanup nahi hua toh timer background mein chalta rahega aur errors aa sakte hain.

SECTION 3: useRef, useMemo, useCallback

11. `useRef` aur `useState` mein kya fark hai?

- A) Koi fark nahi dono same hain
- B) useRef change hone par re-render nahi karta, useState karta hai
- C) useRef sirf DOM elements ke liye hai
- D) useState async hai, useRef sync

✓ Answer: B

■ Why?

useRef `.current` mein value store karta hai jo change hone par component re-render nahi karta. useState re-render trigger karta hai. useRef mutable values store karne ke liye bhi use hota hai.

```
// useRef – re-render nahi karega
const countRef = useRef(0);
countRef.current = 5; // Koi re-render nahi

// useState – re-render karega
const [count, setCount] = useState(0);
setCount(5); // Re-render hoga

// DOM reference example
const inputRef = useRef(null);

```

■ Real Life: Video player mein current time track karna — har second re-render chahiye nahi, useRef perfect hai.

12. `useMemo` ka use kab karna chahiye?

- A) Har jagah jahan calculation ho
- B) Sirf jab expensive calculation ho aur dependencies change na hoon
- C) Event handlers memoize karne ke liye
- D) API calls ke liye

✓ Answer: B

■ Why?

useMemo ek memoized value return karta hai. Use tabhi karo jab calculation genuinely expensive ho (sorting, filtering large arrays). Chhoti calculations pe useMemo lagana overhead add karta hai.

```
const expensiveResult = useMemo(() => {
  // Yeh sirf tab recalculate hoga jab products ya filterText change ho
  return products
  .filter(p => p.name.includes(filterText))
  .sort((a, b) => a.price - b.price);
}, [products, filterText]);

// DON'T do this – unnecessary
const name = useMemo(() => user.firstName + ' ' + user.lastName,
[user]); // Simple string join ke liye useMemo overkill hai
```

■ Real Life: Amazon jaisi site mein 10,000 products filter karna — useMemo se sirf filter text change hone par recalculate hoga.

13. `useCallback` aur `useMemo` mein kya farq hai?

- A) Dono same hain
- B) useCallback function memoize karta hai, useMemo value memoize karta hai
- C) useMemo sirf objects ke liye

D) useCallback sirf event handlers ke liye

✓ **Answer: B**

■ **Why?**

useCallback memoized function reference return karta hai. useMemo memoized value return karta hai. useCallback(fn, deps) === useMemo(() => fn, deps) — technically same hai.

```
// useCallback – function reference stable rakhta hai
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);

// useMemo – computed value memoize karta hai
const sortedList = useMemo(() => {
  return [...list].sort();
}, [list]);

// Child component ko stable function pass karne ke liye
<ExpensiveChild onAction={handleClick} />
// Bina useCallback ke har parent render pe naya function bnega
```

■ **Real Life:** Table mein row delete button — bina useCallback ke har render pe naya function banta hai aur React.memo bhi kaam nahi karta.

14. `useRef` se previous state value kaise store karte hain?

- A) useRef ka koi use nahi is case mein
- B) useEffect mein ref.current ko state se update karo render ke baad
- C) useState se store karo
- D) Directly ref = state likho

✓ **Answer: B**

■ **Why?**

useEffect render ke baad chalta hai. Toh pehle render hoga current value ke saath, phir useEffect mein ref update hoga. Next render mein ref.current mein previous value hogi.

```
function Counter() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef(0);

  useEffect(() => {
    prevCountRef.current = count; // Render ke baad update
  });

  return (
    <div>
      <p>Current: {count}</p>
      <p>Previous: {prevCountRef.current}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>
    </div>
  );
}
```

■ **Real Life:** Stock price app mein pichla price aur current price compare karna — green ya red arrow dikhane ke liye.

15. Ek component mein sirf ek useRef ho sakta hai?

- A) Haan, sirf ek
- B) Nahi, multiple useRef use kar sakte hain
- C) Maximum 3
- D) Ref sirf class components mein use hota hai

✓ **Answer: B**

■ **Why?**

React mein multiple hooks use kar sakte hain. Aap ek component mein kitne bhi useRef, useState, useEffect etc. use kar sakte hain — bas unka order consistent rehna chahiye (conditionally call mat karo).

```
function Form() {
  const nameRef = useRef(null);
  const emailRef = useRef(null);
  const passwordRef = useRef(null);

  function handleSubmit() {
    console.log(nameRef.current.value);
    console.log(emailRef.current.value);
  }
  return (
    <>
    <input ref={nameRef} placeholder="Name" />
    <input ref={emailRef} placeholder="Email" />
    <input ref={passwordRef} type="password" />
    </>
  );
}
```

■ **Real Life:** Multi-field form mein har field ka alag ref — submit pe sab fields ko programmatically access karna.

SECTION 4: useReducer

16. `useReducer` kab prefer karna chahiye `useState` ke upar?

- A) Sirf jab state number ho
- B) Jab state logic complex ho aur multiple sub-values hon ya state transitions rules-based hon
- C) Sirf Redux use karne ke liye
- D) Performance ke liye hamesha

✓ **Answer: B**

■ **Why?**

useReducer better hai jab state complex object ho, multiple state values ek saath update hoon, ya next state pichle state par depend kare. useState simple values ke liye, useReducer complex state machine ke liye.

```
const initialState = { count: 0, loading: false, error: null };

function reducer(state, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 };
    case 'SET_LOADING':
      return { ...state, loading: action.payload };
    case 'SET_ERROR':
      return { ...state, error: action.payload, loading: false };
    default:
```

```
return state;
}
}

const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'INCREMENT' });
```

■ **Real Life:** Shopping cart — items add/remove, quantity update, total calculate sab ek reducer mein manage karo.

17. `useReducer` mein `dispatch` function kya karta hai?

- A) State directly set karta hai
- B) Reducer function ko action ke saath call karta hai aur state update karta hai
- C) Component re-render force karta hai
- D) Async operations handle karta hai

✓ **Answer: B**

■ **Why?**

dispatch ek action object accept karta hai, usse reducer ko bhejta hai. Reducer current state aur action lekar new state return karta hai. React tab component re-render karta hai.

```
// Action dispatch karna
dispatch({ type: 'ADD_ITEM', payload: { id: 1, name: 'Phone' } });
dispatch({ type: 'REMOVE_ITEM', payload: 1 });
dispatch({ type: 'CLEAR_CART' });

// Reducer mein handle
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return { ...state, items: [...state.items, action.payload] };
    case 'REMOVE_ITEM':
      return { ...state,
        items: state.items.filter(i => i.id !== action.payload) };
    case 'CLEAR_CART':
      return { ...state, items: [] };
    default: return state;
  }
}
```

■ **Real Life:** Ek Redux-like pattern bina Redux install kiye — medium apps ke liye useReducer kaafi hota hai.

18. `useReducer` ki initial state lazy initialize kaise karte hain?

- A) Possible nahi
- B) Teesra argument `init` function pass karke
- C) useEffect mein initialize karke
- D) useState ke saath use karke

✓ **Answer: B**

■ **Why?**

useReducer ka teesra argument ek init function hota hai jo initial state compute karta hai. Yeh tab useful hai jab initial state expensive calculation se aaye ya localStorage se load karni ho.

```
function init(initialCount) {
  // Expensive initial state calculation ya localStorage
  const saved = localStorage.getItem('count');
  return { count: saved ? parseInt(saved) : initialCount };
}

const [state, dispatch] = useReducer(
  reducer,
  0, // initialArg – init function ko pass hoga
  init // init function – ek baar hi chalega
);
```

■ **Real Life:** App start hone par saved cart data localStorage se load karna — init function se ek baar load karo.

19. Kya `useReducer` se `dispatch` function ka reference stable hota hai?

- A) Nahi, har render pe naya dispatch milta hai
- B) Haan, React guarantee deta hai dispatch kabhi nahi badlega
- C) Sirf tab jab state nahi badle
- D) useMemo ke saath hi stable hota hai

✓ **Answer: B**

■ **Why?**

React guarantee karta hai ki dispatch function ka reference stable hai — re-renders mein same reference milta hai. Isliye dispatch ko useCallback mein wrap karne ki zaroorat nahi.

```
const [state, dispatch] = useReducer(reducer, initialState);

// dispatch stable hai, useCallback ki zaroorat nahi
// Directly child mein pass kar sakte ho
<ChildComponent onAction={dispatch} />

// Yeh unnecessary hai:
// const stableDispatch = useCallback(dispatch, []); // No need!
```

■ **Real Life:** Context mein dispatch pass karte waqt — stable reference hone se unnecessary re-renders nahi hote children mein.

20. `useState` vs `useReducer` — performance mein kya fark hai?

- A) useReducer hamesha faster hai
- B) useState hamesha faster hai
- C) Performance difference minimal hai, choice architecture par depend karta hai
- D) useReducer memoization karta hai automatically

✓ **Answer: C**

■ **Why?**

Performance mein koi significant difference nahi. useReducer ka benefit code organization aur predictability hai, speed nahi. Complex state ke liye useReducer readable hai.

```
// useState – simple ke liye theek hai
const [isOpen, setIsOpen] = useState(false);
const [title, setTitle] = useState('');

// useReducer – complex state ke liye better
const [modalState, dispatch] = useReducer(modalReducer, {
  isOpen: false,
```

```
title: '',
content: null,
type: 'info'
});
// Ek dispatch mein multiple fields update ho sakti hain
```

■ Real Life: Simple toggle ke liye useState, modal/drawer state (isOpen, title, type, data) ke liye useReducer.

SECTION 5: useTransition & useDeferredValue

21. `useTransition` hook kya problem solve karta hai?

- A) Async API calls ko handle karta hai
- B) UI ko responsive rakhta hai by marking some state updates as non-urgent
- C) CSS transitions add karta hai
- D) Component lazy load karta hai

✓ Answer: B

■ Why?

useTransition urgent vs non-urgent updates differentiate karta hai. Urgent update (typing) immediately hota hai, transition (search results) defer ho jaata hai. Isse UI block nahi hoti.

```
import { useState, useTransition } from 'react';

function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleSearch(e) {
    setQuery(e.target.value); // Urgent – input turant update

    startTransition(() => {
      setResults(searchData(e.target.value)); // Non-urgent
    });
  }
  return (
    <>
    <input value={query} onChange={handleSearch} />
    {isPending ? <p>Searching...</p> : <ResultsList data={results} />}
    </>
  );
}
```

■ Real Life: Google search jaisi live search — typing smooth rehti hai aur results background mein update hote hain.

22. `useDeferredValue` aur `useTransition` mein kya fark hai?

- A) Dono same hain
- B) useDeferredValue value defer karta hai, useTransition state update defer karta hai. useDeferredValue tab use karo jab parent se value aaye.
- C) useDeferredValue sirf strings ke liye
- D) useTransition React 18 mein tha, useDeferredValue React 19 mein naya hai

✓ **Answer: B**

■ **Why?**

useTransition apni state updates ko control karta hai. useDeferredValue kisi bhi value (prop bhi) ko defer kar sakta hai. Dono concurrent features hain.

```
// useTransition – apne state update control karo
const [isPending, startTransition] = useTransition();
startTransition(() => setState(newValue));

// useDeferredValue – prop ya external value defer karo
function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);
  // deferredQuery thoda peeche chalega query se
  const results = useMemo(
    () => searchData(deferredQuery),
    [deferredQuery]
  );
  return <List data={results} />;
}
```

■ **Real Life:** Parent se search query prop milti hai — useDeferredValue se expensive filtering defer karo bina parent modify kiye.

23. `useTransition` se `isPending` flag ka kya matlab hai?

- A) API call chal rahi hai
- B) Transition state update abhi pending hai — complete nahi hua
- C) Component mount ho raha hai
- D) Error aayi hai

✓ **Answer: B**

■ **Why?**

isPending true hota hai jab startTransition ke andar wrapped update abhi complete nahi hua. Is time user ko loading indicator dikhao.

```
const [isPending, startTransition] = useTransition();

// isPending ka use
{isPending && (
  <div className="overlay">
    <Spinner />
  </div>
)}

// Ya opacity se indicate karo
<div style={{ opacity: isPending ? 0.6 : 1 }}>
  <HeavyComponent data={data} />
</div>
```

■ **Real Life:** Tab switching mein — current tab ka content thoda fade karo jab naya tab load ho raha ho.

24. Kya `startTransition` ke andar `async` function use kar sakte hain? (React 19)

- A) Nahi, sirf sync code
- B) Haan, React 19 mein async functions support hai startTransition mein
- C) Sirf useEffect ke saath
- D) Sirf Server Actions ke saath

✓ **Answer: B**

■ **Why?**

React 19 mein startTransition async functions support karta hai. Yeh ek bada improvement hai React 18 se, jahan sirf synchronous callbacks allowed the.

```
// React 19 – async startTransition ■
const [isPending, startTransition] = useTransition();

async function handleSubmit(formData) {
  startTransition(async () => {
    const result = await saveData(formData); // Async kaam karta hai
    setUser(result.user);
    setMessage('Saved!');
  });
}

// isPending await ke dauran bhi true rahega
```

■ **Real Life:** Form save karna — async data save + UI update dono ek saath transition mein handle karo.

25. `useDeferredValue` se kya guarantee milti hai?

- A) Value kabhi stale nahi hogi
- B) Deferred value eventually consistent ho jaayegi urgent updates ke baad
- C) Value 500ms delay ke saath aayegi
- D) Value kabhi update nahi hogi

✓ **Answer: B**

■ **Why?**

useDeferredValue temporary stale value rakh sakta hai. React pehle urgent update render karega, phir jab time mile deferred value ke saath re-render karega. Eventually dono sync ho jaate hain.

```
function ProductList({ searchText }) {
  const deferredSearch = useDeferredValue(searchText);

  // Ye check karo kya stale data dikh raha hai
  const isStale = deferredSearch !== searchText;

  const filtered = useMemo(
    () => products.filter(p => p.name.includes(deferredSearch)),
    [deferredSearch]
  );

  return (
    <ul style={{ opacity: isStale ? 0.7 : 1 }}>
      {filtered.map(p => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}
```

■ **Real Life:** Search results thode dim dikhao jab user type kar raha ho — bata raha hai ki fresh results aa rahe hain.

SECTION 6: useId & useImperativeHandle

26. `useId` hook kab use karte hain?

- A) Unique list keys generate karne ke liye
- B) Accessibility ke liye stable unique IDs banane ke liye jo server aur client mein match karein
- C) User authentication ke liye
- D) Database IDs generate karne ke liye

✓ **Answer: B**

■ **Why?**

`useId` accessibility attributes (`htmlFor`, `aria-labelledby`) ke liye unique, stable IDs generate karta hai. Yeh SSR mein bhi server-client hydration mismatch avoid karta hai. List keys ke liye mat use karo.

```
import { useId } from 'react';

function FormField({ label, type }) {
  const id = useId(); // ':r0:', ':r1:' jaise unique IDs

  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} type={type} />
    </div>
  );
}

// Multiple IDs ek component mein
function ComplexForm() {
  const id = useId();
  return (
    <>
      <input id={`${id}-name`} aria-label="Name" />
      <input id={`${id}-email`} aria-label="Email" />
    </>
  );
}
```

■ **Real Life:** Reusable Input component library banate waqt — har instance ko unique ID chahiye label aur input match karne ke liye.

27. `useImperativeHandle` kya karta hai?

- A) Parent ko child ka poora DOM expose karta hai
- B) Parent ko child ke specific methods/values expose karta hai ref ke through
- C) Child component ko parent ka DOM access deta hai
- D) Sirf class components mein use hota hai

✓ **Answer: B**

■ **Why?**

`useImperativeHandle` se aap control kar sakte ho ki parent ko `ref.current` mein kya milega. Poora DOM expose karne ki jagah sirf specific methods expose karo — encapsulation maintain hoti hai.

```
// React 19 — forwardRef ke bina
function VideoPlayer({ ref }) {
  const videoRef = useRef(null);

  useImperativeHandle(ref, () => ({
    play: () => videoRef.current.play(),
```

```

pause: () => videoRef.current.pause(),
// Poora video DOM expose nahi – sirf ye methods
});

return <video ref={videoRef} src="movie.mp4" />;
}

// Parent mein
const playerRef = useRef(null);
<VideoPlayer ref={playerRef} />
playerRef.current.play(); // Direct DOM nahi, sirf exposed method

```

■ **Real Life:** Custom video player component — parent sirf play/pause call kare, baaki video DOM internally rahe.

28. `useImperativeHandle` kab use karna chahiye?

- A) Har component mein
- B) Jab parent ko child ke specific imperative actions trigger karni hoon (focus, scroll, play)
- C) Performance optimization ke liye hamesha
- D) State management ke liye

✓ **Answer: B**

■ **Why?**

useImperativeHandle sparingly use karo. Sirf tab jab parent ko child ke andar kuch trigger karna ho jo state/props se possible na ho jaise focus, scroll position, animation trigger.

```

function SearchInput({ ref }) {
  const inputRef = useRef(null);

  useImperativeHandle(ref, () => ({
    focus: () => {
      inputRef.current.focus();
      inputRef.current.select(); // Text select bhi
    },
    clear: () => {
      inputRef.current.value = '';
    }
  }));

  return <input ref={inputRef} />;
}

// Parent
const searchRef = useRef();
<button onClick={() => searchRef.current.focus()}>
  Search karo
</button>

```

■ **Real Life:** Search bar mein keyboard shortcut (Ctrl+K) press karne par input focus karna parent component se.

SECTION 7: Custom Hooks

29. Custom hook banana kab zaroori hai?

- A) Jab component 100 lines se bada ho
- B) Jab same stateful logic multiple components mein reuse karni ho
- C) Sirf performance ke liye
- D) Jab useEffect use karna ho

✓ **Answer: B**

■ **Why?**

Custom hooks stateful logic extract karte hain aur reuse karte hain. Agar same useEffect + useState combination 2-3 components mein likha hai, custom hook banao.

```
// Custom hook – useFetch
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then(res => res.json())
      .then(d => { setData(d); setLoading(false); })
      .catch(e => { setError(e); setLoading(false); });
  }, [url]);

  return { data, loading, error };
}

// Use karo kisi bhi component mein
const { data, loading, error } = useFetch('/api/users');
```

■ **Real Life:** Har API call ke liye loading, error, data logic repeat hota tha — useFetch banao ek baar, har jagah use karo.

30. Custom hook ka naam kis se shuru hona chahiye?

- A) 'custom'
- B) 'use'
- C) 'hook'
- D) Kuch bhi chal sakta hai

✓ **Answer: B**

■ **Why?**

'use' prefix React ke liye convention nahi, rule hai. React lint rules check karte hain ki hooks 'use' se shuru hon — isse React jan sakta hai ki yeh ek hook hai aur rules apply kare.

```
// Sahi naming ■
function useLocalStorage(key, initialValue) { ... }
function useDebounce(value, delay) { ... }
function useOnlineStatus() { ... }
function useWindowSize() { ... }

// Galat naming ✗ – React hook rules apply nahi honge
function fetchData() { useState... } // Warning aayega
function localStorageHelper() { useEffect... } // Bug prone
```

■ **Real Life:** ESLint react-hooks plugin 'use' prefix check karta hai — team mein consistency ke liye naming convention follow karo.

31. Custom hook se kya return karna chahiye?

- A) Hamesha array [value, setValue]
- B) Hamesha object {}
- C) Jo bhi useful ho — value, array, object, function, ya kuch nahi
- D) Sirf JSX

✓ Answer: C

■ Why?

Custom hook se kuch bhi return kar sakte ho. useState jaisa array, useQuery jaisa object, ya sirf ek value. Design karo usecase ke hisaab se.

```
// Array return (useState pattern)
const [value, setValue] = useLocalStorage('key', '');

// Object return (multiple values)
const { data, loading, error, refetch } = useFetch(url);

// Single value return
const isOnline = useOnlineStatus();

// Function return
const debouncedValue = useDebounce(searchText, 300);

// Kuch return nahi (sirf side effect)
usePageTitle('My App');
```

■ Real Life: API hook ke liye object return karo (data, loading, error, refetch), toggle hook ke liye [value, toggle] array.

32. `useLocalStorage` custom hook mein kya problem aa sakti hai SSR mein?

- A) Koi problem nahi
- B) Server pe `window` aur `localStorage` available nahi hote — error aayega
- C) localStorage async hai server pe
- D) SSR mein hooks work nahi karte

✓ Answer: B

■ Why?

Next.js jaisi SSR frameworks mein server pe window object nahi hota. localStorage access karne se 'window is not defined' error aata hai. typeof window check karna zaroori hai.

```
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    // SSR check – typeof window undefined check
    if (typeof window === 'undefined') return initialValue;

    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  const setValue = (value) => {
```

```

setStoredValue(value);
if (typeof window !== 'undefined') {
  localStorage.setItem(key, JSON.stringify(value));
}
};
return [storedValue, setValue];
}

```

■ Real Life: Next.js app mein theme preference localStorage mein save karna — SSR check nahi kiya toh build time crash.

SECTION 8: React 19 – Actions & Forms

33. React 19 mein form `action` prop kya accept karta hai?

- A) Sirf string URL
- B) String URL ya async function dono
- C) Sirf POST method ke saath
- D) Sirf Server Components mein

✓ Answer: B

■ Why?

React 19 mein form ka action prop ek async function accept karta hai (Server Action ya client function). React automatically form data pass karta hai aur pending state handle karta hai.

```

// React 19 – function as action ■
async function handleSubmit(formData) {
  'use server'; // Server Action
  const name = formData.get('name');
  await saveUser(name);
}

<form action={handleSubmit}>
  <input name="name" />
  <button type="submit">Save</button>
</form>

// Client side bhi kaam karta hai
async function clientAction(formData) {
  const data = Object.fromEntries(formData);
  await fetch('/api/save', { method: 'POST', body: JSON.stringify(data) });
}

```

■ Real Life: Contact form submit karna — action mein async function dao, React pending state manage karega automatically.

34. `useActionState` hook (React 19) kya karta hai?

- A) useState ka replacement hai
- B) Form action ke state, error, aur pending ko manage karta hai
- C) Redux actions handle karta hai
- D) Sirf server mein use hota hai

✓ Answer: B

■ Why?

useActionState (pehle useFormState tha) form action ka result state, error state aur pending state ek saath manage karta hai. Yeh React 19 ka key form hook hai.

```
import { useActionState } from 'react';

async function submitForm(prevState, formData) {
  const name = formData.get('name');
  if (!name) return { error: 'Name required' };
  await saveUser(name);
  return { success: true };
}

function ContactForm() {
  const [state, action, isPending] = useActionState(
    submitForm,
    { error: null, success: false } // Initial state
  );

  return (
    <form action={action}>
      <input name="name" />
      {state.error && <p className="error">{state.error}</p>}
      {state.success && <p>Saved!</p>}
      <button disabled={isPending}>
        {isPending ? 'Saving...' : 'Save'}
      </button>
    </form>
  );
}
```

■ Real Life: Registration form — validation errors dikhao, loading state handle karo, success message show karo — sab ek hook mein.

35. React 19 mein Server Actions ko client component mein kaise use karte hain?

- A) Direct import se
- B) Props ke through pass karke ya module import se
- C) Sirf server components mein possible hai
- D) fetch() se call karo

✓ Answer: B

■ Why?

Server Actions ko client components mein props ke through ya direct import se use kar sakte hain. 'use server' directive function ke top mein lagao.

```
// actions.js
'use server';
export async function deleteUser(id) {
  await db.users.delete(id);
  revalidatePath('/users');
}

// ClientComponent.jsx
'use client';
import { deleteUser } from './actions';
```

```
function UserCard({ user }) {
  return (
    <div>
      <p>{user.name}</p>
      <form action={deleteUser.bind(null, user.id)}>
        <button type="submit">Delete</button>
      </form>
    </div>
  );
}
```

■ **Real Life:** Next.js app mein user delete karna — Server Action se DB directly access karo, API route banane ki zaroorat nahi.

SECTION 9: React.memo, Lazy Loading & Suspense

36. `React.memo` kya karta hai?

- A) useMemo jaisa hai, values memoize karta hai
- B) Component ko wrap karta hai — sirf tab re-render karta hai jab props change hoon
- C) Component ko lazy load karta hai
- D) Component ki state memoize karta hai

✓ **Answer: B**

■ **Why?**

React.memo Higher-Order Component hai jo component ko shallow prop comparison se re-render optimize karta hai. Agar parent re-render ho lekin props same hon toh child re-render skip karta hai.

```
// Bina memo — har parent render pe re-render hoga
function ExpensiveChild({ name, onClick }) {
  console.log('Child rendered');
  return <button onClick={onClick}>{name}</button>;
}

// Memo ke saath — sirf props change hone par re-render
const ExpensiveChild = React.memo(function({ name, onClick }) {
  console.log('Child rendered');
  return <button onClick={onClick}>{name}</button>;
});

// Custom comparison bhi de sakte ho
const Comp = React.memo(Component, (prevProps, nextProps) => {
  return prevProps.id === nextProps.id; // true = re-render skip
});
```

■ **Real Life:** Data table mein rows — parent filter change kare lekin individual row ka data same ho, toh re-render skip karo.

37. `React.lazy()` aur `Suspense` saath kaise kaam karte hain?

- A) React.lazy() immediately component load karta hai
- B) React.lazy() dynamic import se component late load karta hai, Suspense fallback dikhata hai loading mein
- C) Suspense error handling ke liye hai

D) Dono sirf SSR mein kaam karte hain

✓ **Answer: B**

■ **Why?**

React.lazy() code splitting enable karta hai — component sirf tab load hota hai jab zaroorat ho. Suspense loading ke dauran fallback UI dikhata hai.

```
import { lazy, Suspense } from 'react';

// Component sirf tab load hoga jab render hoga
const Dashboard = lazy(() => import('./Dashboard'));
const Settings = lazy(() => import('./Settings'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      {currentPage === 'dashboard'
        ? <Dashboard />
        : <Settings />
      }
    </Suspense>
  );
}
```

■ **Real Life:** Admin panel — Dashboard, Reports, Settings — sab alag chunks. User sirf Dashboard kholta hai toh Settings ka code download nahi hoga. Bundle size kam.

38. `Suspense` nested karne ka kya faida hai?

A) Koi faida nahi

■ B) Different loading boundaries set kar sakte ho — different parts ke alag fallbacks

C) Performance improve hoti hai automatically

D) Error boundary ka kaam karta hai

✓ **Answer: B**

■ **Why?**

Nested Suspense se different loading granularity milti hai. Header jaldi load ho, sidebar baad mein — dono ke alag loading spinners ho sakte hain.

```
<Suspense fallback={<AppShell />}> { /* Outer */}
<Header /> { /* Fast load */}

<Suspense fallback={<SidebarSkeleton />}> { /* Inner */}
<Sidebar /> { /* Slow load */}
</Suspense>

<Suspense fallback={<ContentSkeleton />}>
  <MainContent /> { /* Slowest */}
</Suspense>
</Suspense>

// Har section apna skeleton dikhata hai
// User header turant dekhta hai
```

■ **Real Life:** News app — navbar turant dikhe, featured articles 1 sec mein, recommended articles 3 sec mein — alag alag skeletons.

39. `ErrorBoundary` Suspense ke saath kab use karte hain?

- A) Kabhi nahi
- B) Jab lazy component load fail ho ya data fetch fail ho — error catch karne ke liye
- C) Sirf development mein
- D) Automatic hota hai

✓ Answer: B

■ Why?

Suspense loading handle karta hai, ErrorBoundary failure handle karta hai. Dono saath use karo — load fail hone ya data error aane par user-friendly message dikhao.

```
class ErrorBoundary extends Component {
  state = { hasError: false };
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError)
      return <p>Kuch problem aayi! Retry karo.</p>;
    return this.props.children;
  }
}
```

```
<ErrorBoundary>
<Suspense fallback={<Spinner />}>
<LazyDashboard />
</Suspense>
</ErrorBoundary>
```

■ Real Life: Network offline hone par lazy component load fail ho — ErrorBoundary se friendly message dikhao, blank screen nahi.

40. React 19 mein `use(promise)` Suspense ke saath kaise kaam karta hai?

- A) Promise manually resolve karna padhta hai
- B) use() promise pending mein Suspense fallback dikhata hai, resolve hone par data milta hai
- C) Sirf rejected promises handle karta hai
- D) use() Suspense ke bina kaam karta hai

✓ Answer: B

■ Why?

use(promise) call karo — jab tak promise pending hai Suspense fallback dikhega. Resolve hone par component render hoga data ke saath. Reject hone par ErrorBoundary catch karega.

```
// Data fetch function (Server Component ya parent mein)
const userPromise = fetch('/api/user').then(r => r.json());
```

```
// Client Component
function UserProfile({ userPromise }) {
  const user = use(userPromise); // Suspends until resolved
  return <h1>Hello, {user.name}!</h1>;
}
```

```
// Parent wrap karo
<ErrorBoundary>
<Suspense fallback={<ProfileSkeleton />}>
```

```
<UserProfile userPromise={userPromise} />
</Suspense>
</ErrorBoundary>
```

■ Real Life: Profile page load karna — skeleton dikhao jab tak data aaye, error page dikhao agar API fail ho.

SECTION 10: React Performance Optimization

41. React mein unnecessary re-renders kab hote hain?

- A) Sirf state change hone par
- B) State ya props change hone par, ya parent re-render hone par (even if props same hoon)
- C) Sirf user interaction par
- D) Sirf useEffect mein

✓ Answer: B

■ Why?

By default React mein parent re-render hone par saare children re-render hote hain, chahe unke props same hoon ya nahi. React.memo, useMemo, useCallback se optimize karo.

```
// Problem – Parent state badli, Child bhi re-render hoga
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <>
    <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
    <Child /> { /* Count se related nahi, par re-render hoga */ }
    </>
  );
}

// Solution ■
const Child = React.memo(function Child() {
  return <div>I don't change!</div>;
});
```

■ Real Life: Dashboard mein notifications badge update hone par pura dashboard re-render na ho — React.memo use karo expensive components ke liye.

42. React DevTools mein performance profiling kaise karte hain?

- A) console.log use karo
- B) React DevTools ke Profiler tab mein Record karo aur flame graph dekho
- C) Chrome Performance tab sirf
- D) useEffect mein Date.now() lagao

✓ Answer: B

■ Why?

React DevTools Profiler exactly batata hai kaunsa component kitna time le raha hai render mein. Flame graph mein wide bars = slow components.

```
// Code mein bhi Profiler wrap kar sakte ho
import { Profiler } from 'react';

function onRender(id, phase, actualDuration) {
```

```
console.log(`${id} ${phase} took ${actualDuration}ms`);
}
```

```
<Profiler id="Dashboard" onRender={onRender}>
<Dashboard />
</Profiler>
```

```
// Browser DevTools mein:
// React DevTools > Profiler > Record > Interact > Stop
// Flame graph mein slow components identify karo
```

■ **Real Life:** App slow lagti hai — Profiler se exact bottleneck dhoondho, guess mat karo. Often ek expensive component hoti hai.

43. `key` prop ka React performance mein kya role hai?

- A) Sirf CSS ke liye
- B) React ko batata hai ki element same hai ya naya — reconciliation optimize karta hai
- C) Accessibility ke liye
- D) Debugging ke liye sirf

✓ **Answer: B**

■ **Why?**

key prop React reconciler ko batata hai kaunsa element kaunsa hai. Wrong key ya index as key use karne se bugs aur performance issues aa sakte hain.

```
// Galat — index as key (reordering pe bugs)
{items.map((item, index) => (
<Item key={index} data={item} /> // Bug prone
))}
```

```
// Sahi — stable unique ID ■
{items.map(item => (
<Item key={item.id} data={item} />
))}
```

```
// Key component reset karne ke liye bhi use hota hai
// Yeh trick component ko completely re-create karta hai
<UserProfile key={userId} userId={userId} />
```

■ **Real Life:** Todo list mein item delete karo — index key use karne se animation aur input state galat element mein shift ho jaati hai.

44. React 19 mein React Compiler kya karta hai?

- A) JSX ko TypeScript mein compile karta hai
- B) Automatically memoization add karta hai — useMemo/useCallback manually likhne ki zaroorat kam ho jaati hai
- C) Server-side code compile karta hai
- D) Bundle size optimize karta hai

✓ **Answer: B**

■ **Why?**

React Compiler (pehle React Forget tha) code analyze karke automatically memoization add karta hai. Aapko har jagah useMemo/useCallback nahi likhna padega — compiler sahi jagah pe khud lagata hai.

```
// React Compiler ke bina – manually optimize
const expensiveValue = useMemo(() => compute(a, b), [a, b]);
const stableHandler = useCallback(() => handle(id), [id]);

// React Compiler ke saath – compiler yeh kaam khud karta hai
// Normal code likho, compiler optimize karega:
const expensiveValue = compute(a, b); // Compiler memoize karega
const stableHandler = () => handle(id); // Compiler stable reference dega
```

■ **Real Life:** React 19 + Compiler use karne wale codebases mein boilerplate memoization code significantly kam ho gaya.

45. Virtualization kab use karni chahiye React mein?

- A) Har list ke liye
- B) Jab list mein 100+ items hon jinhe sab ek saath render karna performance issue create kare
- C) Sirf tables ke liye
- D) Sirf mobile apps mein

✓ **Answer: B**

■ **Why?**

Virtualization sirf visible items DOM mein rakhta hai — 10,000 items mein se sirf 20 screen par dikhte hain, 20 hi DOM mein honge. react-window ya react-virtual use karo.

```
import { FixedSizeList } from 'react-window';

function Row({ index, style }) {
  return (
    <div style={style}>
      Item #{index} – {data[index].name}
    </div>
  );
}

function LargeList() {
  return (
    <FixedSizeList
      height={600} // Container height
      itemCount={10000}
      itemSize={50} // Har row ki height
      width="100%"
    >
      {Row}
    </FixedSizeList>
  );
}
```

■ **Real Life:** Gmail jaisi app mein 5000 emails list — virtualization ke bina page freeze ho jaata, saath mein smooth scroll.

SECTION 11: Context API – Deep Dive

46. Context API kab use karni chahiye aur kab nahi?

- A) Hamesha use karo state ke liye

- B) Global ya widely shared state ke liye (theme, auth, language) — local state ke liye useState prefer karo
- C) Sirf performance ke liye
- D) Redux ki jagah hamesha

✓ **Answer: B**

■ **Why?**

Context genuinely global data ke liye hai — theme, user auth, language/locale. Prop drilling avoid karna zaroori ho tab use karo. Performance-sensitive data ke liye Zustand/Redux consider karo.

```
// Good use case – Auth Context
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = async (credentials) => {
    const u = await loginApi(credentials);
    setUser(u);
  };
  return (
    <AuthContext value={{ user, login, logout }}>
      {children}
    </AuthContext>
  );
}

// Bad use case – local form state
// Form state context mein mat daalo – useState use karo
```

■ **Real Life:** Nav bar, sidebar, page header — sab mein logged-in user ka naam dikhana. Auth context perfect use case.

47. Context mein value object hone par performance optimize kaise karte hain?

- A) Koi optimization needed nahi
- B) useMemo se context value memoize karo taaki unnecessary re-renders na hon
- C) useCallback use karo
- D) Context split karo sirf

✓ **Answer: B**

■ **Why?**

Agar context value object hai aur provider ka parent re-render hota hai, toh har baar naya object banta hai. Saare consumers re-render honge. useMemo se object stabilize karo.

```
function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const [preferences, setPreferences] = useState({});

  // useMemo se object stable rakho
  const value = useMemo(() => ({
    user,
    preferences,
    updateUser: setUser,
    updatePrefs: setPreferences
  }), [user, preferences]);
```

```
return (  
  <UserContext value={value}>  
    {children}  
  </UserContext>  
);  
}
```

■ **Real Life:** Frequent re-renders avoid karne ke liye — user preferences update hone par sirf relevant consumers re-render hoon.

48. Context split karne ka kya faida hai?

- A) Koi faida nahi
- B) Different update frequencies alag karne se unnecessary re-renders kam hote hain
- C) Code complicated ho jaata hai
- D) Sirf large apps mein useful hai

✓ **Answer: B**

■ **Why?**

Ek bade context mein agar koi ek value change ho toh saare consumers re-render karte hain. Context split karne se sirf related consumers re-render hote hain.

```
// Galat – ek bada context  
const AppContext = createContext({ user, theme, cart, language });  
  
// Sahi – split contexts ■  
const UserContext = createContext(null);  
const ThemeContext = createContext('light');  
const CartContext = createContext([]);  
  
// Cart update hone par sirf CartContext consumers re-render honge  
// Theme aur User consumers affect nahi honge
```

■ **Real Life:** Theme change aur cart update alag-alag — theme consumers aur cart consumers independently update hon.

SECTION 12: Portals & Error Boundaries

49. `ReactDOM.createPortal` kab use karte hain?

- A) Sirf performance ke liye
- B) Jab component ko DOM hierarchy se bahar render karna ho (modals, tooltips, dropdowns)
- C) Server rendering ke liye
- D) Lazy loading ke liye

✓ **Answer: B**

■ **Why?**

Portal component ko parent DOM ke bahar render karta hai. Modal/tooltip/dropdown ko body ke direct child mein render karo taaki z-index aur overflow:hidden issues na aayein.

```
import { createPortal } from 'react-dom';  
  
function Modal({ isOpen, children, onClose }) {  
  if (!isOpen) return null;
```

```

return createPortal(
  <div className="modal-overlay" onClick={onClose}>
    <div className="modal-content" onClick={e => e.stopPropagation()}>
      {children}
    </div>
  </div>,
  document.body // DOM mein #root ke bahar, body mein render
);
}

// Use karo normally
<Modal isOpen={showModal} onClose={() => setShowModal(false)}>
  <h2>Hello from Portal!</h2>
</Modal>

```

■ Real Life: Table ke andar dropdown menu — bina portal overflow:hidden se clip ho jaata, portal se body mein render karo.

50. Error Boundary class component kyun hona chahiye, function component nahi?

- A) Function components better hain
- B) `getDerivedStateFromError` aur `componentDidCatch` abhi tak sirf class components mein available hain
- C) Hooks error boundaries support karte hain
- D) Performance ke liye

✓ Answer: B

■ Why?

React 18/19 mein abhi bhi Error Boundary sirf class components se banate hain kyunki `getDerivedStateFromError` lifecycle sirf class mein hai. React team future mein hook-based solution la sakti hai.

```

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    // Error logging service mein bhejo
    logErrorToService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return <ErrorFallback error={this.state.error} />;
    }
    return this.props.children;
  }
}

```

■ Real Life: Payment gateway component crash ho — Error Boundary se graceful fallback dikhao, puri app crash na ho.

SECTION 13: React 19 Asset Loading APIs

51. React 19 mein `preload` aur `preinit` kya hain?

- A) CSS frameworks hain
- B) Browser ko resources preload karne ke hints dene ke liye React 19 ke naye APIs hain
- C) Lazy loading APIs hain
- D) Image optimization ke liye

✓ Answer: B

■ Why?

React 19 mein resource loading APIs aaye hain — preload, preinit, prefetchDNS etc. Yeh browser ko pehle se resources fetch karne ke liye keh sakte hain.

```
import { preload, preinit } from 'react-dom';

function App() {
  preinit('https://example.com/script.js', {
    as: 'script' // Script preload + execute
  });

  preload('https://fonts.googleapis.com/css2?family=Inter', {
    as: 'style' // CSS preload (execute nahi)
  });

  preload('/api/user-data', {
    as: 'fetch' // API data prefetch
  });

  return <Dashboard />;
}
```

■ Real Life: App ke critical resources pehle se load karo — fonts, scripts — page LCP (Largest Contentful Paint) improve hoti hai.

52. React 19 mein title, meta aur link tags component mein kaise manage karte hain?

- A) Sirf index.html mein likhte hain
- B) React 19 mein directly component mein likhne par React automatically document head mein move karta hai
- C) react-helmet library zaroori hai
- D) Next.js sirf yeh kar sakta hai

✓ Answer: B

■ Why?

React 19 mein Document Metadata Support aaya. Aap component ke andar , etc. likh sakte ho — React khud inhe mein move kar dega. react-helmet ki zaroorat nahi.

```
// React 19 ■ – react-helmet nahi chahiye
function BlogPost({ post }) {
  return (
    <article>
      <title>{post.title} | My Blog</title>
      <meta name="description" content={post.excerpt} />
      <meta property="og:image" content={post.image} />
    </article>
  );
}
```

```
<link rel="canonical" href={` /blog/${post.slug}`} />

<h1>{post.title}</h1>
<p>{post.content}</p>
</article>
);
}
// React yeh sab automatically <head> mein daal dega
```

■ Real Life: SEO ke liye dynamic page titles — har blog post ka alag title aur description, React manage karega.

SECTION 14: Hooks Rules & Common Patterns

53. Hooks ke bare mein kaunsa rule sabse important hai?

- A) Hooks sirf class components mein use karo
- B) Hooks ko conditionally ya loops mein call mat karo — hamesha top level par call karo
- C) Ek component mein sirf 3 hooks use karo
- D) Hooks ko export karna zaroori hai

✓ Answer: B

■ Why?

React hooks ke call order par depend karta hai state track karne ke liye. Agar conditional mein hook call karo toh order change ho sakta hai aur React state galat component se associate karega.

```
// GALAT ✗
function Component({ showData }) {
  if (showData) {
    const [data, setData] = useState(null); // Conditional! Bug!
  }
  useEffect(() => {}, []); // Is effect ki ordering ab unreliable hai
}

// SAHI ■
function Component({ showData }) {
  const [data, setData] = useState(null); // Hamesha call
  useEffect(() => {
    if (!showData) return; // Logic andar
  }, [showData]);
  if (!showData) return null; // Rendering conditional
}
```

■ Real Life: ESLint eslint-plugin-react-hooks yeh automatically catch karta hai — project mein zaroor lagao.

54. Kya hooks ko nested functions ya callbacks mein call kar sakte hain?

- A) Haan, koi problem nahi
- B) Nahi — hooks sirf function component ke top level mein ya custom hooks mein call hone chahiye
- C) Sirf useEffect ke andar
- D) Sirf async functions mein

✓ Answer: B

■ Why?

Hooks component ke direct body mein call karo — event handlers, timeouts, nested functions mein nahi. React ka rules of hooks yahi kehta hai.

```
// GALAT ✗
function Component() {
  const handleClick = () => {
    const [count, setCount] = useState(0); // Hooks in event handler!
  };

  setTimeout(() => {
    useEffect(() => {}); // Hooks in setTimeout!
  }, 1000);
}

// SAHI ■
function Component() {
  const [count, setCount] = useState(0); // Top level
  useEffect(() => {
    // Logic yahan, hook nahi
    const timer = setTimeout(() => setCount(c => c + 1), 1000);
    return () => clearTimeout(timer);
  }, []);
}
```

■ Real Life: ESLint immediately yeh error detect karta hai — 'Rules of Hooks' violation.

55. `useEffect` mein missing dependency warning kab aati hai?

- A) Jab dependency array empty ho
- B) Jab effect ke andar use ki gayi values dependency array mein listed na hoon
- C) Jab async function use karo
- D) Jab cleanup function na ho

✓ Answer: B

■ Why?

React Hook mein jo bhi values use kar rahe ho (state, props, functions) sab dependency array mein honi chahiye. Warna stale closure bug aa sakta hai — effect purani value use karega.

```
// Warning – userId dependency missing
useEffect(() => {
  fetchUser(userId); // userId dependency array mein nahi
}, []); // Warning: 'userId' missing from deps

// Sahi ■
useEffect(() => {
  fetchUser(userId);
}, [userId]); // userId add kiya

// Functions ke liye – useCallback se stabilize karo
const fetchUser = useCallback(async () => {
  const data = await api.getUser(userId);
  setUser(data);
}, [userId]);

useEffect(() => {
  fetchUser();
}, [fetchUser]); // Stable reference
```

■ Real Life: User ID change hone par user data refetch nahi hua — missing dependency bug, ek common React bug.

56. `useLayoutEffect` aur `useEffect` mein kya fark hai?

- A) Koi fark nahi
- B) useLayoutEffect DOM update ke baad lekin browser paint se pehle synchronously chalta hai
- C) useLayoutEffect server pe bhi chalta hai
- D) useLayoutEffect cleanup support nahi karta

✓ Answer: B

■ Why?

useLayoutEffect DOM mutations ke baad synchronously fire hota hai — browser kuch bhi screen par paint karne se pehle. Isse visual flickering avoid hoti hai DOM measurements ke case mein.

```
// useEffect – paint ke BAAD (async)
useEffect(() => {
  // Screen already paint ho chuki hai
  element.style.color = 'red'; // Flicker dikh sakta hai
}, []);

// useLayoutEffect – paint se PEHLE (sync) ■
useLayoutEffect(() => {
  // Paint se pehle DOM measure/modify karo
  const height = element.getBoundingClientRect().height;
  setHeight(height); // No flicker
}, []);
```

■ Real Life: Tooltip position calculate karna — element ki height measure karo paint se pehle, warna tooltip flash karega galat jagah pe.

SECTION 15: Advanced Patterns & Misc Concepts

57. Render Props pattern kya hai?

- A) Component ka default rendering
- B) Ek technique jahan component ek function prop accept karta hai jo render karne ki responsibility deta hai
- C) Server-side rendering
- D) Redux ka pattern

✓ Answer: B

■ Why?

Render Props ek pattern hai jahan component ek function prop (usually children ya render) lete hain. Yeh logic share karne ka purana tarika hai — custom hooks ne isse mostly replace kar diya.

```
// Render Props pattern
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  return (
    <div onMouseMove={e => setPosition({ x: e.clientX, y: e.clientY })}>
      {render(position)} {/* Function call karke render karo */}
    </div>
  );
}
```

```

}

// Use
<MouseTracker render={({ x, y }) => (
  <p>Mouse position: {x}, {y}</p>
)} />

// Modern equivalent – custom hook
const position = useMousePosition(); // Cleaner!

```

■ **Real Life:** Purani libraries (React Router v4, React Motion) render props use karti thi — ab mostly hooks preferred hain.

58. Compound Component pattern kya hota hai?

- A) Multiple files mein component banana
- B) Parent component ki state share karne wale related components ka set — Context use karke
- C) HOC ka dusra naam
- D) Component inheritance

✓ **Answer: B**

■ **Why?**

Compound Components related components ka group hote hain jo internally Context share karte hain. Select/Option, Tabs/Tab, Accordion/AccordionItem jaisi UI components is pattern se banti hain.

```

const TabContext = createContext();

function Tabs({ children, defaultTab }) {
  const [activeTab, setActiveTab] = useState(defaultTab);
  return (
    <TabContext value={{ activeTab, setActiveTab }}>
      {children}
    </TabContext>
  );
}

function Tab({ id, children }) {
  const { activeTab, setActiveTab } = use(TabContext);
  return (
    <button
      onClick={() => setActiveTab(id)}
      className={activeTab === id ? 'active' : ''}
    >{children}</button>
  );
}

// Usage – Flexible!
<Tabs defaultTab="home">
  <Tab id="home">Home</Tab>
  <Tab id="profile">Profile</Tab>
</Tabs>

```

■ **Real Life:** Custom Tab component library — Headless UI, Radix UI yahi pattern use karte hain.

59. Higher-Order Component (HOC) kya hai?

- A) Sirf class components ke liye
- B) Ek function jo component le aur enhanced component return kare
- C) React Router ka feature
- D) Performance tool

✓ **Answer: B**

■ **Why?**

HOC ek function hai jo component input mein leta hai aur naya component return karta hai extra functionality ke saath. Pattern ab purana hai, custom hooks zyada prefer hain.

```
// HOC – withAuth
function withAuth(Component) {
  return function AuthenticatedComponent(props) {
    const { user } = useAuth();
    if (!user) return <Redirect to="/login" />;
    return <Component {...props} user={user} />;
  };
}

// Use karo
const ProtectedDashboard = withAuth(Dashboard);
const ProtectedSettings = withAuth(Settings);

// Modern equivalent – custom hook
function Dashboard() {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  return <DashboardContent user={user} />;
}
```

■ **Real Life: Authentication, logging, analytics** — HOC se cross-cutting concerns add karo multiple components mein.

60. `React.StrictMode` kya karta hai production mein?

- A) Errors double hoti hain
- B) StrictMode sirf development mein kaam karta hai, production mein koi effect nahi
- C) Performance improve karta hai
- D) TypeScript enable karta hai

✓ **Answer: B**

■ **Why?**

StrictMode development-only tool hai. Yeh components ko double render karta hai side effects detect karne ke liye, deprecated APIs warn karta hai. Production build mein completely inactive hota hai.

```
// index.js / main.jsx
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// Development mein StrictMode:
// 1. Components 2x render karta hai (side effect bugs dhundhne ke liye)
// 2. Effects 2x run karte hain (mount → unmount → remount)
// 3. Deprecated lifecycle methods warn karta hai
```

```
// 4. Unexpected side effects identify karta hai  
  
// Production mein: Koi difference nahi, zero overhead
```

■ **Real Life:** `useEffect` cleanup theek se nahi likha — `StrictMode` double mount karne par bug immediately visible ho jaata hai.

61. `useDebugValue` kab use karte hain?`

- A) `Console.log` ka replacement
- B) Custom hooks mein — React DevTools mein hook ka debug label dikhane ke liye
- C) Error handling ke liye
- D) Performance measure karne ke liye

✓ **Answer: B**

■ **Why?**

`useDebugValue` custom hooks mein use hota hai taaki React DevTools mein hook ka meaningful label dikhe. Library authors ke liye useful hai.

```
function useOnlineStatus() {  
  const [isOnline, setIsOnline] = useState(navigator.onLine);  
  
  useDebugValue(isOnline ? '■ Online' : '■ Offline');  
  // DevTools mein: "useOnlineStatus: ■ Online" dikhhega  
  
  useEffect(() => {  
    const handleOnline = () => setIsOnline(true);  
    const handleOffline = () => setIsOnline(false);  
    window.addEventListener('online', handleOnline);  
    window.addEventListener('offline', handleOffline);  
    return () => {  
      window.removeEventListener('online', handleOnline);  
      window.removeEventListener('offline', handleOffline);  
    };  
  }, []);  
  return isOnline;  
}
```

■ **Real Life:** NPM package mein custom hook publish karo — users DevTools mein easily debug kar sakein.

62. React mein `key` prop change karne par kya hota hai?`

- A) Component update hota hai
- B) Component completely unmount aur re-mount hota hai — fresh state ke saath
- C) Sirf re-render hota hai
- D) Koi effect nahi

✓ **Answer: B**

■ **Why?**

key change hone par React purana component destroy karta hai aur bilkul naya mount karta hai. Yeh ek deliberate reset trick hai — component ki state completely clear ho jaati hai.

```
// Key trick – component state reset karo  
function UserProfile({ userId }) {  
  return (  
    // userId change hone par poori ProfileForm fresh ho jaayegi
```

```

<ProfileForm key={userId} userId={userId} />
);
}

// Iske bina – ProfileForm old user ka state hold kar sakti hai
// Iske baad – naya userId = fresh component = clean state

// Controlled reset button bhi implement kar sakte ho
const [formKey, setFormKey] = useState(0);
<Form key={formKey} />
<button onClick={() => setFormKey(k => k + 1)}>Reset Form</button>

```

■ **Real Life:** User profile edit karo — ek user se doosre user switch karo, form fields cleanly reset ho jaayein.

63. Controlled vs Uncontrolled components mein kya fark hai?

A) Koi fark nahi

■ B) Controlled — React state form data manage karta hai. Uncontrolled — DOM khud manage karta hai (useRef se access)

C) Uncontrolled better performance dete hain hamesha

D) Controlled sirf class components mein hote hain

✓ **Answer: B**

■ **Why?**

Controlled component mein value prop aur onChange React ko dete ho — React source of truth hai. Uncontrolled mein DOM apna state rakhta hai — ref se value read karo.

```

// Controlled ■ (recommended)
const [name, setName] = useState('');
<input value={name} onChange={e => setName(e.target.value)} />

// Uncontrolled (kabhi kabhi simple cases mein ok)
const nameRef = useRef();
<input ref={nameRef} defaultValue="Raj" />
// nameRef.current.value – submit par read karo

// Controlled prefer karo:
// - Validation on type
// - Dependent fields
// - Disable button jab form invalid ho

```

■ **Real Life:** Signup form — controlled use karo taaki real-time password strength check, field validation, submit button enable/disable sab kuch possible ho.

64. `defaultProps` React 19 mein kya hua?

A) Still recommended hai

■ B) Function components mein deprecated hai — default parameters use karo

C) Completely remove ho gaya

D) Sirf TypeScript mein use karo

✓ **Answer: B**

■ **Why?**

React 19 mein function components ke liye defaultProps deprecated mark kiya gaya. JavaScript default parameters use karo — cleaner aur idiomatic JS hai.

```
// Deprecated in React 19 ✗
function Button({ label, color }) { ... }
Button.defaultProps = { color: 'blue', label: 'Click' };

// Modern way ■ – JS default parameters
function Button({ label = 'Click', color = 'blue', size = 'md' }) {
  return (
    <button style={{ backgroundColor: color }} className={size}>
      {label}
    </button>
  );
}

// TypeScript ke saath
interface Props {
  label?: string;
  color?: string;
}
function Button({ label = 'Click', color = 'blue' }: Props) { ... }
```

■ **Real Life:** Component library mein default props — JavaScript defaults zyada readable hain aur tree-shaking friendly bhi.

65. React mein `reconciliation` kya hai?

- A) State management process
- B) Virtual DOM aur real DOM compare karke minimum changes apply karna
- C) Server-side rendering process
- D) Context update karna

✓ **Answer: B**

■ **Why?**

Reconciliation React ka algorithm hai — naya Virtual DOM purane se compare karta hai (diffing) aur sirf zaruri DOM changes apply karta hai. Yahi React ki efficiency ka secret hai.

```
// React Virtual DOM diff karta hai
// Before:
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>

// After state change:
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Cherry</li> // Sirf yeh naya DOM node add hoga
</ul>

// Key ka role – list reordering mein
// Bina key – React sab re-render karta hai
// Saath key – React sirf moved/added items handle karta hai
```

■ **Real Life:** 100 item list mein ek item add karo — reconciliation se sirf ek DOM node add hoga, 100 re-render nahi honge.

66. `React.Fragment` kab use karte hain?

- A) Sirf performance ke liye
- B) Multiple elements return karne ke liye bina extra DOM node banaye
- C) Context provider ki jagah
- D) Sirf lists mein

✓ **Answer: B**

■ **Why?**

JSX mein ek root element return karna zaroori hai. Fragment (<> ya) se multiple elements wrap kar sakte ho bina actual DOM element banaye.

```
// Galat – extra div DOM mein add hoti hai
return (
  <div> {/* Unnecessary div */}
  <h1>Title</h1>
  <p>Content</p>
</div>
);

// Sahi – Fragment, no extra DOM ■
return (
  <>
  <h1>Title</h1>
  <p>Content</p>
</>
);

// Key ke saath – full syntax use karo
return (
  <React.Fragment key={item.id}>
  <dt>{item.term}</dt>
  <dd>{item.description}</dd>
</React.Fragment>
);
```

■ **Real Life: Table cells** — ke andar extra add karna invalid HTML hai, Fragment use karo.

67. React mein `children` prop kya hota hai?

- A) Array of components sirf
- B) Component ke opening aur closing tags ke beech ka content — string, element, array, function kuch bhi ho sakta hai
- C) Sirf text content
- D) Props ka subset

✓ **Answer: B**

■ **Why?**

children ek special prop hai jo component tags ke beech likha content hota hai. Yeh string, number, JSX element, array, ya function ho sakta hai.

```
function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="body">{children}</div>
    </div>
  );
}
```

```

}

// Different children types
<Card title="Simple">Hello!</Card> // String child

<Card title="Complex">
  <p>Paragraph</p>
  <button>Click</button>
</Card> // JSX children

// children ko manipulate karo
React.Children.map(children, child =>
  React.cloneElement(child, { extraProp: true })
);

```

■ **Real Life:** Layout components (Card, Modal, Sidebar) — children prop se flexible content accept karo.

68. `React.cloneElement` kab use karte hain?

- A) Component copy karne ke liye
- B) Existing element ko clone karke extra props inject karne ke liye
- C) Component memoize karne ke liye
- D) Dynamic import ke liye

✓ **Answer: B**

■ **Why?**

cloneElement existing React element ka copy banata hai naye props ke saath merge karke. Compound components aur render props patterns mein use hota hai.

```

function Toolbar({ children }) {
  return (
    <div className="toolbar">
      {React.Children.map(children, child =>
        React.cloneElement(child, {
          // Har child button ko theme prop inject karo
          theme: 'dark',
          size: 'sm'
        })
      )}
    </div>
  );
}

<Toolbar>
  <Button>Save</Button> // theme='dark', size='sm' mil jaayega
  <Button>Cancel</Button> // theme='dark', size='sm' mil jaayega
</Toolbar>

```

■ **Real Life:** Form component — har child input ko required prop automatically inject karo, har input mein manually likhne ki zaroorat nahi.

69. React mein synthetic events kya hain?

- A) Custom DOM events
- B) React ka cross-browser wrapper native browser events ke upar — consistent API deta hai
- C) TypeScript events

D) Async events

✓ **Answer: B**

■ **Why?**

React Synthetic Events native browser events ko wrap karte hain taaki cross-browser consistency mile. React 17+ mein events document se root element par delegate hote hain.

```
function Form() {
  function handleSubmit(e) {
    e.preventDefault(); // Same API cross-browser
    e.stopPropagation();
    console.log(e.target); // Form DOM node
    console.log(e.nativeEvent); // Original browser event
  }

  function handleClick(e) {
    // e ek SyntheticEvent hai
    // e.type, e.target, e.currentTarget – standard properties
    console.log(e.type); // 'click'
  }
  return <form onSubmit={handleSubmit}><button onClick={handleClick}></form>;
}
```

■ **Real Life:** Form submit prevent karna — `e.preventDefault()` cross-browser same kaam karta hai, React handle karta hai.

70. React mein `PropTypes` ka kya role hai?

A) TypeScript ka replacement

■ B) Runtime type checking — development mein props ki type validate karta hai warnings ke saath

C) Performance optimization

D) State type checking

✓ **Answer: B**

■ **Why?**

PropTypes development mein props ki type check karta hai. Galat prop type milne par console warning aata hai. TypeScript compile-time check karta hai — TypeScript zyada powerful hai.

```
import PropTypes from 'prop-types';

function UserCard({ name, age, isAdmin, onClick }) {
  return <div onClick={onClick}>{name} ({age})</div>;
}

UserCard.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  isAdmin: PropTypes.bool,
  onClick: PropTypes.func
};

UserCard.defaultProps = {
  age: 0,
  isAdmin: false
};

// TypeScript use kar rahe ho toh PropTypes ki zaroorat nahi
```

```
interface Props { name: string; age?: number; }
```

■ Real Life: Team mein JavaScript use kar rahe ho — PropTypes se documentation aur runtime safety milti hai bina TypeScript ke.

SECTION 16: Popular React Hook Packages

71. React mein hooks install karne ki zaroorat kab hoti hai?

A) Hamesha npm se install karo

■ B) React ke built-in hooks (useState, useEffect etc.) install nahi karte — react package mein hain. Extra libraries sirf advanced use cases ke liye install karte hain.

C) Har project mein react-use install karo

D) Hooks alag package mein hain

✓ Answer: B

■ Why?

React ke core hooks (useState, useEffect, useContext, useRef, useMemo, useCallback, useReducer, useId, useTransition etc.) sab react package mein built-in hain. Sirf third-party hooks libraries (react-query, zustand etc.) install karni padti hain.

```
// Built-in hooks – install ki zaroorat nahi
import { useState, useEffect, useContext,
  useRef, useMemo, useCallback,
  useReducer, useId, useTransition,
  useDeferredValue, useOptimistic,
  useImperativeHandle, useLayoutEffect,
  useDebugValue } from 'react';

// react-dom se (install nahi chahiye separately)
import { useFormStatus } from 'react-dom';

// Extra libraries – install karni padti hain
// npm install @tanstack/react-query
// npm install zustand
// npm install react-hook-form
```

■ Real Life: Interview mein puchha — 'hooks install karne padte hain?' — Answer: Built-in nahi, third-party libraries haan.

72. `@tanstack/react-query` (React Query) kya problem solve karta hai?

A) State management

■ B) Server state management — data fetching, caching, background refetch, loading/error states automatically

C) Form handling

D) Routing

✓ Answer: B

■ Why?

React Query server state (API data) manage karta hai. Caching, background sync, loading states, error handling — sab automatic. useEffect + useState se manually karna iska kaam zyada tedious hota hai.

```
// npm install @tanstack/react-query

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(r => r.json()),
    staleTime: 5 * 60 * 1000, // 5 min cache
  });

  if (isLoading) return <Spinner />;
  if (error) return <Error />;
  return <UserList users={data} />;
}

// useMutation – create/update/delete ke liye
const mutation = useMutation({
  mutationFn: (user) => fetch('/api/users', {
    method: 'POST', body: JSON.stringify(user)
  }),
  onSuccess: () => queryClient.invalidateQueries(['users'])
});
```

■ **Real Life: E-commerce product list** — auto refresh, background sync, pagination, infinite scroll — React Query se sab easily milta hai.

73. `Zustand` kya hai aur React Context se kaise alag hai?

A) Context ka dusra naam

■ B) Minimal state management library — Context se simple, Redux se simpler, performance better

C) Form management library

D) React ka official tool

✓ **Answer: B**

■ **Why?**

Zustand ek simple external state management library hai. Context ke unlike aap sirf subscribed state ka part change hone par re-render hote ho — zyada performant. Redux se much less boilerplate.

```
// npm install zustand
import { create } from 'zustand';

const useCartStore = create((set, get) => ({
  items: [],
  total: 0,
  addItem: (item) => set(state => ({
    items: [...state.items, item],
    total: state.total + item.price
  })),
  removeItem: (id) => set(state => ({
    items: state.items.filter(i => i.id !== id)
  })),
}));

// Any component mein – no Provider needed!
function Cart() {
```

```
const items = useCartStore(state => state.items); // Sirf items par subscribe
const addItem = useCartStore(state => state.addItem);
return <div>{items.length} items</div>;
}
```

■ Real Life: Shopping cart global state — Zustand se sirf 20-30 lines mein complete solution, Redux ke 100+ lines ki jagah.

74. `react-hook-form` kya faida deta hai?

A) Forms ke liye CSS library

■ B) Uncontrolled inputs par based form management — less re-renders, better performance, validation built-in

C) Server forms ke liye

D) TypeScript forms ke liye sirf

✓ Answer: B

■ Why?

react-hook-form uncontrolled inputs use karta hai — sirf submit par ya specific validation trigger par re-render hota hai. Yeh 100 field forms mein bhi fast rahta hai.

```
// npm install react-hook-form
import { useForm } from 'react-hook-form';

function LoginForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting }
  } = useForm();

  const onSubmit = async (data) => {
    await loginUser(data); // data = { email, password }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input
        {...register('email', {
          required: 'Email required',
          pattern: { value: /^[^\s]+@[^\s]+$/i, message: 'Invalid email' }
        })}
      />
      {errors.email && <p>{errors.email.message}</p>}
      <button disabled={isSubmitting}>Login</button>
    </form>
  );
}
```

■ Real Life: Multi-step form wizard — react-hook-form se state, validation, errors sab manage karo efficiently.

75. `SWR` library kya hai?

A) State management library

■ B) Stale-While-Revalidate — data fetching hook library by Vercel, cached data pehle dikhao phir background mein fresh data fetch karo

- C) CSS-in-JS library
- D) Testing library

✓ **Answer: B**

■ **Why?**

SWR (Stale-While-Revalidate) caching strategy follow karta hai — pehle cached data dikhao (fast), phir network se fresh data fetch karo. React Query se simpler API.

```
// npm install swr
import useSWR from 'swr';

const fetcher = (url) => fetch(url).then(r => r.json());

function Profile() {
  const { data, error, isLoading, mutate } = useSWR(
    '/api/user',
    fetcher,
    {
      refreshInterval: 30000, // 30 sec auto refresh
      revalidateOnFocus: true // Tab focus par refresh
    }
  );

  if (isLoading) return <Spinner />;
  if (error) return <p>Error!</p>;
  return (
    <div>
      <p>{data.name}</p>
      <button onClick={() => mutate()}>Refresh</button>
    </div>
  );
}
```

■ **Real Life:** Dashboard mein live stats — SWR se stale data turant dikhe, background mein fresh data aata rahe.

SECTION 17: React 19 – More Deep Dive

76. React 19 mein `ref` cleanup function kya hai?

- A) Possible nahi
- B) Ref callback se ek function return kar sakte hain jo element DOM se remove hone par call hoga
- C) useEffect mein ref cleanup likho
- D) Sirf class components mein

✓ **Answer: B**

■ **Why?**

React 19 mein ref callbacks ek cleanup function return kar sakte hain — bilkul useEffect ki tarah. Jab component unmount ho ya ref change ho, cleanup call hoga.

```
// React 19 – Ref Cleanup ■
function Input() {
  return (
    <input
      ref={(node) => {
```

```

if (node) {
  // Mount — setup karo
  const observer = new ResizeObserver(() => {
    console.log('Input resized!');
  });
  observer.observe(node);

  // Cleanup function return karo ■ (React 19 naya!)
  return () => {
    observer.disconnect();
    console.log('Cleanup!');
  };
}
}
/>
);
}

```

■ Real Life: Custom input par ResizeObserver lagao — cleanup function se observer automatically remove hoga unmount par.

77. React 19 mein `hydration` errors kaise improve hue hain?

- A) Koi improvement nahi
- B) React 19 better error messages deta hai — exact diff dikhata hai server aur client HTML ka
- C) Hydration errors ab warnings hain
- D) Hydration React 19 mein nahi hoti

✓ Answer: B

■ Why?

React 19 hydration errors mein much better diff output deta hai — exactly batata hai kaunsa element server aur client pe different tha. Debugging zyada easy ho gayi.

```

// React 18 error (confusing):
// "Hydration failed because the initial UI does not match
// what was rendered on the server"

// React 19 error (detailed) ■:
// Error: Hydration mismatch
// Server: <div class="light">
// Client: <div class="dark">
// ^^^^^ Exact difference!

// Common hydration mismatch causes:
// 1. typeof window check nahi kiya
// 2. Date.now() ya Math.random() server/client alag
// 3. Browser extensions HTML modify karte hain
// 4. suppressHydrationWarning prop use karo dynamic content ke liye

```

■ Real Life: Next.js app mein theme localStorage se read karo — server pe 'light', client pe 'dark' hydration mismatch. suppressHydrationWarning lagao.

78. `useOptimistic` mein rollback kaise hota hai?

- A) Manually karna padta hai
- B) Jab async action fail ho toh React automatically original state restore karta hai

- C) Rollback possible nahi
- D) useReducer se handle karo

✓ **Answer: B**

■ **Why?**

useOptimistic automatic rollback provide karta hai. Jab async operation fail ho, React optimistic state discard karta hai aur original state wapas aa jaati hai.

```
function TodoList({ todos, onAddTodo }) {
  const [optimisticTodos, addOptimisticTodo] = useOptimistic(
    todos,
    (currentTodos, newTodo) => [...currentTodos, newTodo]
  );

  async function handleAdd(text) {
    const tempTodo = { id: 'temp', text, pending: true };
    addOptimisticTodo(tempTodo); // Turant UI update

    try {
      await saveTodo(text); // Server call
      // Success – actual data parent se aayegi (refetch)
    } catch (error) {
      // Failure – React automatically optimistic state rollback karega!
      alert('Save failed, rolling back...');
    }
  }

  return <ul>{optimisticTodos.map(t => <li key={t.id}>{t.text}</li>)}</ul>;
}
```

■ **Real Life:** Todo add karo — network fail ho toh todo disappear ho jaata hai automatically. No manual rollback code needed.

79. React 19 mein `createRoot` mein kya new options aaye?

- A) Koi change nahi
- B) `onCaughtError`, `onUncaughtError`, `onRecoverableError` callbacks aaye hain better error handling ke liye
- C) createRoot deprecated ho gaya
- D) Async createRoot

✓ **Answer: B**

■ **Why?**

React 19 mein createRoot mein error handling callbacks aaye — onCaughtError (ErrorBoundary caught errors), onUncaughtError (uncaught errors), onRecoverableError (automatic recovery).

```
import { createRoot } from 'react-dom/client';

const root = createRoot(document.getElementById('root'), {
  onCaughtError(error, errorInfo) {
    // ErrorBoundary ne catch kiya
    logToSentry(error, { componentStack: errorInfo.componentStack });
  },
  onUncaughtError(error, errorInfo) {
    // Kisi ErrorBoundary ne nahi pakda
    showCrashDialog(error);
    logCriticalError(error);
  },
});
```

```
onRecoverableError(error) {  
  // React ne recover kar liya (hydration mismatch etc.)  
  console.warn('Recoverable error:', error);  
}  
});  
  
root.render(<App />);
```

■ **Real Life:** Production mein Sentry ya LogRocket mein React errors log karna — ab zyada granular control milta hai.

80. React mein `Concurrent Mode` ka React 19 mein kya status hai?

- A) Still experimental
- B) React 18 se default hai, React 19 mein aur mature — createRoot use karne par automatic
- C) Deprecated ho gaya
- D) Manual enable karna padhta hai

✓ **Answer: B**

■ **Why?**

Concurrent features React 18 mein stable hue, React 19 mein aur improve hue. createRoot use karte hi concurrent mode active ho jaata hai — opt-in nahi, default hai.

```
// React 16/17 (legacy mode)  
ReactDOM.render(<App />, root); // No concurrent features  
  
// React 18+ / React 19 (concurrent mode) ■  
const root = createRoot(document.getElementById('root'));  
root.render(<App />);  
// Ab sab concurrent features available:  
// - useTransition  
// - useDeferredValue  
// - Suspense improvements  
// - Automatic batching  
// - useOptimistic (React 19)  
// - Async actions (React 19)
```

■ **Real Life:** CRA ya Vite se naya React project — already concurrent mode mein hai. createRoot default use karo.

SECTION 18: React Testing Basics

81. React components test karne ke liye sabse popular library kaunsi hai?

- A) Enzyme (Airbnb)
- B) React Testing Library (@testing-library/react)
- C) Jest only
- D) Cypress only

✓ **Answer: B**

■ **Why?**

React Testing Library (RTL) ab industry standard hai. Yeh user perspective se test karta hai — implementation details nahi. 'Test the way users use your app' philosophy.

```
// npm install @testing-library/react @testing-library/jest-dom

import { render, screen, fireEvent } from '@testing-library/react';

test('Counter increments on click', () => {
  render(<Counter />);

  const button = screen.getByRole('button', { name: /increment/i });
  const count = screen.getByText(/count: 0/i);

  fireEvent.click(button);
  expect(screen.getByText(/count: 1/i)).toBeInTheDocument();
});

test('Shows error for invalid email', async () => {
  render(<LoginForm />);
  fireEvent.change(screen.getByLabelText(/email/i), {
    target: { value: 'invalid' }
  });
  fireEvent.click(screen.getByRole('button', { name: /submit/i }));
  expect(await screen.findByText(/invalid email/i)).toBeVisible();
});
```

■ **Real Life: Login form test** — user ki tarah email type karo, submit karo, error message dhundho — implementation nahi.

82. `act()` React testing mein kab use karte hain?

- A) Sirf async tests mein
- B) Jab state updates aur DOM changes properly flush karni hon test mein
- C) Mocking ke liye
- D) Import karne ke liye

✓ **Answer: B**

■ **Why?**

act() ensure karta hai ki sab state updates aur effects test assertions se pehle complete ho jaayein. RTL internally act() use karta hai — manually kum hi lagana padta hai.

```
import { act, render } from '@testing-library/react';

// RTL's fireEvent aur userEvent automatically act() use karte hain
// Manually tab lagao jab manual state updates trigger karo

test('Timer updates', async () => {
  render(<TimerComponent />);

  await act(async () => {
    // Fake timers advance karo
    jest.advanceTimersByTime(1000);
  });

  expect(screen.getByText('1 second passed')).toBeInTheDocument();
});

// @testing-library/user-event prefer karo fireEvent ke upar
import userEvent from '@testing-library/user-event';
```

```
await userEvent.click(button); // More realistic
```

■ Real Life: Async form submit test — act ke bina test race condition mein fail ho sakta hai.

83. React mein hooks ko test kaise karte hain directly?

- A) Hooks test nahi ho sakte
- B) `@testing-library/react-hooks` ya `renderHook` from RTL use karo
- C) Component wrap karke sirf
- D) Jest directly call karta hai hooks

✓ Answer: B

■ Why?

renderHook (React Testing Library mein built-in) se custom hooks directly test kar sakte ho bina wrapper component ke.

```
import { renderHook, act } from '@testing-library/react';

// Custom hook test
test('useCounter increments', () => {
  const { result } = renderHook(() => useCounter(0));

  expect(result.current.count).toBe(0);

  act(() => {
    result.current.increment();
  });

  expect(result.current.count).toBe(1);
});

test('useFetch returns data', async () => {
  const { result } = renderHook(() => useFetch('/api/users'));

  // Initially loading
  expect(result.current.loading).toBe(true);

  await act(async () => {
    await new Promise(resolve => setTimeout(resolve, 0));
  });

  expect(result.current.data).toBeDefined();
});
```

■ Real Life: useFetch custom hook test — mock fetch karo, loading state, data, error sab verify karo.

SECTION 19: React Router v6/v7 Basics

84. React Router v6 mein `useNavigate` kaise use karte hain?

- A) useHistory jo v5 mein tha
- B) useNavigate hook se navigate function milta hai jo programmatic navigation karta hai
- C) window.location.href sirf
- D) Link component se

✓ **Answer: B**

■ **Why?**

React Router v6 mein useHistory ko useNavigate ne replace kiya. navigate('/path') se programmatic navigation, navigate(-1) se back, navigate(1) se forward.

```
import { useNavigate } from 'react-router-dom';

function LoginForm() {
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    try {
      await login(formData);
      navigate('/dashboard'); // Login ke baad redirect
    } catch (err) {
      navigate('/error', { state: { message: err.message } });
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      { /* ... */ }
      <button onClick={() => navigate(-1)}>Go Back</button>
      <button type="submit">Login</button>
    </form>
  );
}
```

■ **Real Life:** Login success hone par dashboard redirect, logout hone par home redirect — useNavigate se karo.

85. React Router mein `useParams` kya return karta hai?

- A) Query string parameters
- B) URL dynamic parameters object — jaise :id, :slug
- C) Navigation state
- D) Route props

✓ **Answer: B**

■ **Why?**

useParams URL ke dynamic segments return karta hai. Route mein :productId define karo, useParams() se { productId: '123' } milega.

```
// Route definition
<Route path="/products/:productId" element={<ProductDetail />} />
<Route path="/blog/:year/:slug" element={<BlogPost />} />

// Component mein
function ProductDetail() {
  const { productId } = useParams();
  const { data } = useFetch(`/api/products/${productId}`);
  return <div>{data?.name}</div>;
}

function BlogPost() {
  const { year, slug } = useParams();
  // URL: /blog/2024/react-19-guide
```

```
// year = '2024', slug = 'react-19-guide'  
return <div>Post: {slug} ({year})</div>;  
}
```

■ Real Life: Product detail page — URL mein /products/456 se 456 extract karo useParams se aur API call karo.

86. React Router v6 mein nested routes kaise define karte hain?

- A) Sirf parent mein route define karo
- B) Parent Route ke andar child Routes + `` use karo child render karne ke liye
- C) useNested() hook
- D) Nested routing v6 mein possible nahi

✓ Answer: B

■ Why?

React Router v6 mein nested routes parent component mein Outlet component ke zariye render hote hain. Parent component ka layout maintain rehta hai.

```
// Route setup  
<Routes>  
  <Route path="/dashboard" element={<DashboardLayout />}>  
    <Route index element={<Overview />} /> { /* /dashboard */ }  
    <Route path="users" element={<Users />} /> { /* /dashboard/users */ }  
    <Route path="settings" element={<Settings />} />  
  </Route>  
</Routes>  
  
// DashboardLayout.jsx  
function DashboardLayout() {  
  return (  
    <div>  
      <Sidebar /> { /* Common sidebar */ }  
      <main>  
        <Outlet /> { /* Child routes yahan render honge */ }  
      </main>  
    </div>  
  );  
}
```

■ Real Life: Admin panel — sidebar + header common layout, content area mein alag-alag pages nested routes se.

SECTION 20: Mixed Concepts – Final Round

87. React mein `batch updates` kya hain?

- A) Multiple renders ko ek mein combine karna
- B) React 18+ mein multiple setState calls ek re-render mein batch hote hain — unnecessary re-renders avoid
- C) API calls batch karna
- D) Sirf event handlers mein hota hai

✓ Answer: B

■ Why?

React 18 mein Automatic Batching aaya — ab async callbacks (setTimeout, promises) mein bhi multiple setState ek re-render mein batch hote hain. React 17 mein sirf event handlers mein hota tha.

```
// React 17 – async mein batching nahi tha (2 re-renders)
setTimeout(() => {
  setCount(c => c + 1); // Re-render 1
  setFlag(f => !f); // Re-render 2
}, 1000);

// React 18/19 – Automatic Batching ■ (1 re-render)
setTimeout(() => {
  setCount(c => c + 1); // Batch
  setFlag(f => !f); // Batch
}, 1000); // Sirf 1 re-render!

// Batching rokni ho toh:
import { flushSync } from 'react-dom';
flushSync(() => setCount(c => c + 1)); // Immediate render
flushSync(() => setFlag(f => !f)); // Immediate render
```

■ Real Life: API response mein 3 state updates — React 18 se automatically ek re-render mein hote hain, performance better.

88. `React.lazy` ke saath dynamic import ka syntax kya hai?

- A) import(Component)
- B) () => import('./Component')
- C) lazy(import('./Component'))
- D) require('./Component')

✓ Answer: B

■ Why?

React.lazy ek function accept karta hai jo promise return kare — dynamic import syntax () => import('./Component') exactly yahi hai.

```
// Sahi syntax ■
const Dashboard = React.lazy(() => import('./Dashboard'));
const Settings = React.lazy(() => import('./pages/Settings'));

// Named export lazy load karna
const { Chart } = React.lazy(() =>
import('./Chart').then(module => ({ default: module.Chart })))
);

// Route-based code splitting
const routes = [
  { path: '/home', Component: lazy(() => import('./pages/Home')) },
  { path: '/about', Component: lazy(() => import('./pages/About')) },
];
```

■ Real Life: Large app mein route-based code splitting — pehli load par sirf Home page ka JS aaye, Settings page visit karne par Settings JS load ho.

89. React mein `Portal` mein event bubbling kaise kaam karta hai?

- A) Portal mein events bubble nahi karte
- B) Portal DOM hierarchy se bahar ho lekin React tree hierarchy se bubble karta hai — parent component mein events milte hain

- C) Sirf DOM hierarchy follow karta hai
- D) Events block ho jaate hain

✓ **Answer: B**

■ **Why?**

Portal DOM mein body ke under render ho lekin React component tree mein parent ke under rehta hai. Events React tree se bubble karte hain — DOM structure se nahi.

```
function Parent() {  
  function handleClick() {  
    // YEH CALL HOGA even though Modal is in document.body!  
    console.log('Parent caught modal click');  
  }  
  
  return (  
    <div onClick={handleClick}>  
      <Modal> { /* document.body mein render hoga */ }  
      <button>Click me!</button>  
    </Modal>  
    { /* Click Modal mein hoga, React tree se Parent tak bubble karega */ }  
  </div>  
);  
}
```

■ **Real Life:** Modal mein click hone par parent ki onClick trigger ho sakti hai — e.stopPropagation() zaroori ho sakta hai.

90. React mein `useEffect` ke andar async function kaise likhte hain?

- A) useEffect(async () => {}, []) — directly async
- B) Andar ek async function define karo aur call karo — ya IIFE use karo
- C) useAsync() hook use karo
- D) Promise.then() sirf

✓ **Answer: B**

■ **Why?**

useEffect callback async nahi ho sakta kyunki async function Promise return karta hai lekin React cleanup function expect karta hai ya undefined. Andar async function define karo.

```
// GALAT – useEffect async ✗  
useEffect(async () => {  
  const data = await fetchData(); // React warning!  
  setData(data);  
}, []);  
  
// SAHI – andar async function ■  
useEffect(() => {  
  async function loadData() {  
    try {  
      const data = await fetchData();  
      setData(data);  
    } catch (error) {  
      setError(error);  
    }  
  }  
  loadData();  
}, []);
```

```
// Ya IIFE
useEffect(() => {
  (async () => {
    const data = await fetchData();
    setData(data);
  })();
}, []);
```

■ Real Life: Component mount hone par API call — async/await inside useEffect ka sahi pattern.

91. `useCallback` kab actually helpful nahi hota?

- A) Hamesha helpful hai
- B) Jab child component React.memo use nahi karta — useCallback ka koi faida nahi
- C) Sirf development mein helpful hai
- D) Large applications mein

✓ Answer: B

■ Why?

useCallback ka faida sirf tab hai jab stable function reference matter kare — React.memo wrapped child ya useEffect dependency. Bina memo ke child ko stable reference se koi fark nahi.

```
// useCallback ka koi faida nahi yahan
function Parent() {
  const handleClick = useCallback(() => {
    doSomething();
  }, []); // Useless – no benefit

  return <Child onClick={handleClick} />;
}

// Child React.memo nahi hai – har parent render pe re-render hoga

// useCallback USEFUL hai yahan ■
const MemoChild = React.memo(Child); // Memo use kiya
function Parent() {
  const handleClick = useCallback(() => {
    doSomething(id);
  }, [id]); // Now useful – MemoChild won't re-render unnecessarily
  return <MemoChild onClick={handleClick} />;
}
```

■ Real Life: React.memo + useCallback ek team hai — dono saath lagao, sirf ek se kaam nahi.

92. React mein `Object.is` comparison kab relevant hai?

- A) Sirf TypeScript mein
- B) React state comparisons mein — useState/useEffect dependencies Object.is se compare hoti hain
- C) Sirf objects ke liye
- D) Performance profiling mein

✓ Answer: B

■ Why?

React state change detect karne ke liye Object.is use karta hai. Same value set karne par re-render nahi hota. Objects/arrays ke liye reference check hota hai — new object = new reference = re-render.

```
// Object.is comparison examples:
Object.is(1, 1) // true – same primitive
Object.is({}, {}) // false – different references
Object.is(NaN, NaN) // true – special case

// React mein:
const [count, setCount] = useState(0);
setCount(0); // Object.is(0, 0) = true, no re-render ■

const [user, setUser] = useState({ name: 'Raj' });
setUser({ name: 'Raj' }); // Object.is({}, {}) = false, RE-RENDER!
// Naya object = new reference = React re-renders

// Solution:
setUser(prev => prev.name === 'Raj' ? prev : { name: 'Raj' });
// Same reference return karo agar kuch change nahi hua
```

■ **Real Life:** State mein same value set karne ki wajah se unexpected re-renders — Object.is samjho to debug karo.

93. React 19 mein `` action aur React Server Actions ka kya connection hai?

- A) Koi connection nahi
- B) Form action prop mein Server Action function pass karo — directly server par run hoga, API route nahi chahiye
- C) Sirf Next.js mein
- D) Server Actions deprecated hain

✓ **Answer: B**

■ **Why?**

React 19 + Next.js/frameworks mein Server Actions directly form action mein pass hote hain. Server par execute hote hain — no API route needed. 'use server' directive se mark karo.

```
// app/actions.js
'use server';
import { revalidatePath } from 'next/cache';

export async function createPost(formData) {
  const title = formData.get('title');
  const content = formData.get('content');

  await db.posts.create({ title, content });
  revalidatePath('/posts'); // Next.js cache invalidate
}

// app/create-post/page.jsx (Server Component)
import { createPost } from '../actions';

export default function CreatePostPage() {
  return (
    <form action={createPost}> {/* Server Action! */}
    <input name="title" required />
    <textarea name="content" required />
    <button type="submit">Create Post</button>
    </form>
  );
}
```

```
}
```

■ Real Life: Blog post create karna — Server Action se directly DB mein save karo, no REST API endpoint banane ki zaroorat.

94. React DevTools se kya-kya inspect kar sakte hain?

- A) Sirf component names
- B) Component tree, props, state, hooks values, context, profiling, renders count
- C) Sirf console errors
- D) Network requests

✓ Answer: B

■ Why?

React DevTools browser extension bahut powerful hai — component tree, har component ke props/state/hooks, context values, render counts, profiler se performance analysis.

```
// React DevTools mein:  
// 1. Components tab:  
// - Component tree dekho  
// - Props edit karo live  
// - State inspect karo  
// - Hooks values dekho (useState, useReducer values)  
// - Context values  
  
// 2. Profiler tab:  
// - Record session  
// - Flame graph — kaunsa component slow hai  
// - Render count aur duration  
// - Why component re-rendered  
  
// Useful: Components tab > Filter > Search component name  
// Click component > Right panel mein props + state
```

■ Real Life: Component ki state kya hai debug karna — DevTools mein directly dekho bina console.log ke.

95. React mein `flushSync` kab use karte hain?

- A) Hamesha
- B) Jab React batching bypass karni ho aur immediate synchronous DOM update chahiye
- C) Performance ke liye
- D) Server rendering mein

✓ Answer: B

■ Why?

flushSync React ki normal batching bypass karta hai aur synchronous DOM update force karta hai. Rarely use karo — sirf tab jab immediately updated DOM chahiye hoon next line mein.

```
import { flushSync } from 'react-dom';  
  
function ScrollToBottom() {  
  const listRef = useRef(null);  
  
  function addItem() {  
    flushSync(() => {  
      setItems(prev => [...prev, newItem]); // Turant DOM update
```

```
});
// Yahan DOM already updated hai
// Toh scroll correctly bottom tak jaayega
listRef.current.scrollTop = listRef.current.scrollHeight;
}
return <ul ref={listRef}>{items.map(...)}</ul>;
}

// Bina flushSync – setItems batch hota, scroll purani height se hota
```

■ **Real Life:** Chat app mein naya message add karo aur automatically scroll to bottom — flushSync ensure karta hai scroll sahi height pe ho.

96. React 19 mein `Suspense` list ka kya improvement hua?

- A) Koi change nahi
- B) List ke items gradually reveal ho sakte hain — SuspenseList jaisi functionality improve hui
- C) Suspense deprecated hua
- D) Sirf images ke liye

✓ **Answer: B**

■ **Why?**

React 19 mein Suspense aur transitions saath use karne se better list reveal experiences possible hain. Pehle sab ek saath aate the, ab staggered reveals possible hain.

```
// Suspense + Transition = Smooth list loading
function ProductGrid() {
  const [category, setCategory] = useState('all');
  const [isPending, startTransition] = useTransition();

  return (
    <>
      <CategoryFilter
        onChange={cat => startTransition(() => setCategory(cat))}
      />
      <div style={{ opacity: isPending ? 0.7 : 1 }}>
        <Suspense fallback={<GridSkeleton />}>
          <ProductList category={category} />
        </Suspense>
      </div>
    </>
  );
}
// Filter change karne par: dim grid, new products load, reveal
```

■ **Real Life:** E-commerce category filter — purana category dim ho, naye products skeleton ke saath load hon, smooth transition.

97. React mein `prop drilling` kya hai aur solutions kya hain?

- A) Props TypeScript mein type karna
- B) Props ko bahut deeply nested components tak pass karna — solution: Context, composition, ya state management
- C) Props validate karna
- D) Default props set karna

✓ **Answer: B**

■ **Why?**

Prop drilling tab hota hai jab intermediate components ko props pass karne padte hain sirf aage forward karne ke liye. Solutions: Context API, component composition, Zustand/Redux.

```
// Prop drilling problem
function App() {
  const [user, setUser] = useState(userObj);
  return <Layout user={user}><Main user={user} /></Layout>;
}
function Layout({ user, children }) {
  return <>{children}<Header user={user} /></>;
}
function Header({ user }) {
  return <Avatar user={user} />; // Sirf Avatar ko chahiye ■■
}

// Solution 1 – Context ■
const UserContext = createContext();
<UserContext value={user}><App /></UserContext>
// Avatar mein: const user = use(UserContext);

// Solution 2 – Composition ■
function App() {
  return <Layout header={<Header user={user} />} />;
}
```

■ **Real Life:** User avatar component ko 5 levels niche pass karna — Context se seedha access karo.

98. React mein `Controlled Input` mein cursor jump problem kab hoti hai aur fix kya hai?

A) Cursor kabhi jump nahi karta

■ B) Jab value prop asynchronously update ho ya case normalize ho — solution: state sync rakhna

C) Sirf mobile mein

D) TypeScript fix karta hai

✓ **Answer: B**

■ **Why?**

Controlled input mein agar value re-render ke beech change ho ya string normalize ho (uppercase), cursor end pe jump kar sakta hai. State synchronously update karke fix hota hai.

```
// Problem – cursor jumps
function Input() {
  const [value, setValue] = useState('');
  // Async state update = cursor jump
  async function handleChange(e) {
    const result = await someAsyncTransform(e.target.value);
    setValue(result); // React state update
  }
  return <input value={value} onChange={handleChange} />;
}

// Fix ■ – Synchronous update
function Input() {
  const [value, setValue] = useState('');
  function handleChange(e) {
    const transformed = e.target.value.toUpperCase();
    setValue(transformed);
  }
  return <input value={value} onChange={handleChange} />;
}
```

```
setValue(transformed); // Sync – no cursor jump
}
return <input value={value} onChange={handleChange} />;
}
```

■ Real Life: Uppercase conversion input — sync karo value update, cursor position preserve rahegi.

99. React mein `reconciler` aur `renderer` mein kya fark hai?

- A) Same cheez hain
- B) Reconciler changes dhundhta hai (virtual DOM diff), renderer actual output pe apply karta hai (DOM, Native, etc.)
- C) Reconciler sirf server mein hai
- D) Renderer React ka purana naam tha

✓ Answer: B

■ Why?

React ka architecture two parts mein hai: Reconciler (react package) changes calculate karta hai, Renderer (react-dom, react-native, etc.) unhe target platform par apply karta hai. Isliye ek codebase web aur native dono pe kaam karta hai.

```
// Reconciler – react package
// - Virtual DOM maintain karta hai
// - Diff algorithm chalata hai
// - Kya change karna hai batata hai

// Renderers – different platforms
// react-dom – Browser DOM update karta hai
// react-native – iOS/Android native views
// react-three-fiber – Three.js 3D rendering
// react-pdf – PDF generate karta hai
// ink – Terminal/CLI rendering

// Yahi reason hai ki:
import { useState } from 'react'; // Reconciler
import { createRoot } from 'react-dom/client'; // Browser renderer
import { AppRegistry } from 'react-native'; // Native renderer
```

■ Real Life: React Native — same React components iOS aur Android par run hote hain, sirf renderer alag hota hai.

100. Interview mein pucha: React 19 ka sabse important naya feature kya hai apke opinion mein aur kyun?

- A) forwardRef remove karna
- B) Actions aur async transitions — form handling aur loading states ka complexity dramatically reduce hua, real-world apps ke liye most impactful
- C) useId hook
- D) React.memo improvement

✓ Answer: B

■ Why?

Actions (form action as function, useActionState, useFormStatus, useOptimistic) saath mein ek complete story dete hain async operations ke liye. Loading states, optimistic updates, error handling — sab built-in. Yeh web apps ki most common patterns ko significantly simplify karta hai.

```

// React 19 Actions – Complete pattern
'use server';
export async function submitOrder(prevState, formData) {
  try {
    const order = await createOrder(formData);
    return { success: true, orderId: order.id };
  } catch {
    return { error: 'Order failed. Try again.' };
  }
}

// Client
function OrderForm() {
  const [state, action, isPending] = useActionState(submitOrder, null);
  const [optimisticMsg, setOptimisticMsg] = useOptimistic(null);

  return (
    <form action={action}>
      {state?.error && <Alert>{state.error}</Alert>}
      {state?.success && <Success id={state.orderId} />}
      <SubmitButton /> { /* useFormStatus internally */ }
    </form>
  );
}

```

■ Real Life: E-commerce checkout — order submit, loading state, success/error — pehle 50+ lines tha, React 19 mein 20 lines. Yahi sabse bada win hai.